Go Language

200 Interview Q&A — SDE2 Edition

SDE2 Interview Preparation Series

iPad Edition · Dark Code Blocks · Syntax Highlighting

Go (Golang) SDE2 Interview Guide — 200 Questions & Answers

> **Topics:** Language Fundamentals, Concurrency, Runtime Internals, Memory, Performance, Testing, Patterns, HTTP/gRPC, Go + DB/Kafka | **Level:** SDE2

Table of Contents

---

1. Language Fundamentals

Q1

What are Go's key design principles?

Difficulty: Easy

Go is designed for simplicity, readability, and fast compilation. Key principles: explicit over implicit, composition over inheritance, concurrency as a first-class feature, fast build times, single binary output, garbage collection, and strong standard library.

```go
// Explicit error handling (no exceptions)
f, err := os.Open("file.txt")
if err != nil {
    return fmt.Errorf("open file: %w", err)
}
defer f.Close()

// Composition over inheritance
type Animal struct{ Name string }
type Dog struct {
    Animal          // embedded (composition)
    Breed string
}
d := Dog{Animal: Animal{Name: "Rex"}, Breed: "Labrador"}
fmt.Println(d.Name) // promoted field
```

Q2

What is the difference between var, :=, and const?

Difficulty: Easy

```go
// var: explicit declaration, zero value if no initializer
var x int        // x = 0
var s string     // s = ""
var b bool       // b = false

// := short declaration: infers type, only inside functions
x := 42
s := "hello"
m := map[string]int{"a": 1}

// const: compile-time constant, cannot be changed
const Pi = 3.14159
const MaxRetries = 3
const (
    StatusOK    = 200
    StatusNotFound = 404
)

// iota: auto-incrementing const
type Direction int
const (
    North Direction = iota // 0
    East                   // 1
    South                  // 2
    West                   // 3
)
```

Q3

What are Go's basic data types?

Difficulty: Easy

```go
// Integers
int, int8, int16, int32, int64
uint, uint8, uint16, uint32, uint64
uintptr  // pointer-sized uint

// Floats
float32, float64  // always use float64

// Complex
complex64, complex128

// String: immutable, UTF-8 bytes
s := "hello"
s[0] = 'H'  // COMPILE ERROR: strings are immutable

// byte = uint8, rune = int32 (Unicode code point)
var b byte = 'A'
var r rune = '■'

// Boolean
var ok bool = true

// Zero values
var i int     // 0
var f float64 // 0.0
var s string  // ""
var p *int    // nil
var sl []int  // nil
var m map[string]int // nil
```

Q4

What is the difference between arrays and slices?

Difficulty: Medium

```go
// Array: fixed size, value type (copied on assignment)
var a [3]int = [3]int{1, 2, 3}
b := a       // b is a COPY
b[0] = 99
fmt.Println(a[0]) // 1 (original unchanged)

// Slice: dynamic size, reference type (shares backing array)
s := []int{1, 2, 3}
t := s       // t shares same backing array
t[0] = 99
fmt.Println(s[0]) // 99 (original changed!)

// Slice internals: pointer + length + capacity
s := make([]int, 3, 5) // len=3, cap=5
fmt.Println(len(s), cap(s)) // 3 5

// append may allocate new backing array
s = append(s, 4, 5)    // fits in cap
s = append(s, 6)       // new backing array allocated!

// Safe slice copy
dst := make([]int, len(src))
copy(dst, src)

// Slice tricks
s[2:5]      // reslice [2,5)
s[:3]       // from start to index 3
s[2:]       // from index 2 to end
s[:]        // copy of header, shares backing array
```

Q5

How do maps work in Go?

Difficulty: Medium

```go
// Map: hash table, reference type
m := make(map[string]int)
m["age"] = 25

// Literal initialization
m := map[string][]string{
    "fruits":  {"apple", "banana"},
    "veggies": {"carrot"},
}

// Safe read (comma-ok idiom)
val, ok := m["key"]
if !ok {
    // key does not exist
}

// Delete
delete(m, "key")

// Iterate (random order!)
for k, v := range m {
    fmt.Printf("%s: %d\n", k, v)
}

// Map is NOT safe for concurrent use
// Use sync.RWMutex or sync.Map for concurrent access

// Nil map: reads return zero value, writes PANIC
var m map[string]int
fmt.Println(m["key"]) // 0 (ok)
m["key"] = 1          // PANIC: assignment to nil map

// Check map length
fmt.Println(len(m))
```

Q6

What is a struct and how does embedding work?

Difficulty: Medium

```go
type Address struct {
    Street string
    City   string
    Zip    string
}

type User struct {
    ID      int64
    Name    string
    Email   string
    Address          // embedded (anonymous field)
    Role    string
}

u := User{
    ID:    1,
    Name:  "Alice",
    Address: Address{Street: "123 Main", City: "NYC"},
}

// Promoted fields: access directly
fmt.Println(u.City)    // same as u.Address.City
fmt.Println(u.Address.City) // explicit

// Embedding is NOT inheritance
// u is NOT an Address; Address's methods are promoted to User
// If User defines its own City(), it shadows Address.City()

// Multiple embeddings
type AdminUser struct {
    User           // gets all User fields
    Permissions []string
}
```

Q7

What are pointers in Go?

Difficulty: Medium

```go
// Pointer: holds memory address of a value
x := 42
p := &x       // p is *int, holds address of x
fmt.Println(*p) // dereference: 42
*p = 100
fmt.Println(x)  // 100

// new(): allocate zero value, return pointer
p := new(int)   // *int pointing to 0
*p = 42

// When to use pointers:
// 1. Mutate the original value
func increment(n *int) { *n++ }

// 2. Large struct: avoid copying
func process(u *User) { ... }

// 3. Optional/nullable value
type Config struct {
    Timeout *time.Duration  // nil = not set
}

// Pointer to struct: fields via . (no -> like C)
u := &User{Name: "Alice"}
u.Name = "Bob"  // same as (*u).Name = "Bob"

// Go has NO pointer arithmetic
// No dangling pointers (GC manages memory)
```

Q8

What is the difference between value receiver and pointer receiver?

Difficulty: Medium

```go
type Counter struct{ count int }

// Value receiver: operates on a COPY
func (c Counter) Value() int {
    return c.count
}

// Pointer receiver: operates on original
func (c *Counter) Increment() {
    c.count++
}

c := Counter{}
c.Increment()     // Go auto-takes address: (&c).Increment()
fmt.Println(c.Value()) // 1

// Rules:
// Use pointer receiver when:
//   1. Method needs to mutate the receiver
//   2. Receiver is a large struct (avoid copy)
//   3. Consistency: if any method needs pointer, use pointer for all

// Value receiver when:
//   1. Receiver is small, immutable (int, string, small struct)
//   2. Method is a pure function (no mutation)

// Interface satisfaction:
// *Counter satisfies interface with both value and pointer receivers
// Counter satisfies ONLY interface with value receivers
var i interface{ Value() int } = Counter{}  // ok
var i interface{ Increment() } = Counter{}  // FAIL: need *Counter
```

Q9

What is defer and how does it work?

Difficulty: Medium

```go
// defer: executes when surrounding function returns
func readFile(path string) error {
    f, err := os.Open(path)
    if err != nil { return err }
    defer f.Close()  // runs when readFile returns (even on panic)

    // ... read file
    return nil
}

// defer is LIFO (stack)
func example() {
    defer fmt.Println("first defer")   // prints last
    defer fmt.Println("second defer")  // prints second
    defer fmt.Println("third defer")   // prints first
    fmt.Println("function body")
}
// Output:
// function body
// third defer
// second defer
// first defer

// defer captures arguments at time of defer call, NOT execution
x := 10
defer fmt.Println(x)  // captures 10, NOT current value when function returns
x = 20                // too late for defer

// Named return values with defer (can modify return)
func divide(a, b float64) (result float64, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered: %v", r)
        }
    }()
    return a / b, nil
}

// defer in loop: AVOID (all defers execute at function return, not loop iteration)
for _, f := range files {
    defer f.Close()  // BAD: all close at function end
}
// Fix: use anonymous function
for _, f := range files {
    f := f  // capture loop variable (pre Go 1.22)
    func() {
        defer f.Close()
        process(f)
    }()
}
```

Q10

How does range work?

Difficulty: Easy

```go
// range over slice: index, value
for i, v := range []int{10, 20, 30} {
    fmt.Printf("index=%d value=%d\n", i, v)
}

// range over map: key, value (RANDOM ORDER)
for k, v := range map[string]int{"a": 1, "b": 2} {
    fmt.Printf("%s=%d\n", k, v)
}

// range over string: index (byte), rune (Unicode)
for i, r := range "Hello, ■" {
    fmt.Printf("%d: %c (%d)\n", i, r, r)
}

// range over channel: receives until closed
for v := range ch {
    fmt.Println(v)
}

// Ignore index or value
for _, v := range slice { }  // ignore index
for i := range slice { }     // ignore value

// IMPORTANT: range copies the value
// Modifying v does NOT modify original slice element
for _, v := range users {
    v.Name = "x"  // does NOT modify users slice
}
// Fix: use index
for i := range users {
    users[i].Name = "x"  // modifies original
}

// Loop variable capture (Go < 1.22 bug)
var funcs []func()
for _, v := range []int{1, 2, 3} {
    v := v  // shadow: create new variable per iteration
    funcs = append(funcs, func() { fmt.Println(v) })
}
// Go 1.22+: loop variable captured correctly by default
```

Q11

What is the blank identifier (_)?

Difficulty: Easy

```go
// Ignore a return value
_, err := fmt.Println("hello")

// Ignore map key or index
for _, v := range slice { }
for k := range m { }  // value ignored

// Import for side effects only
import _ "net/http/pprof"  // registers pprof handlers

// Ignore struct fields in assignment
x, _, z := multiReturn()

// Check interface implementation at compile time
var _ io.Reader = (*MyReader)(nil)
// If *MyReader doesn't implement io.Reader → compile error

// Ignore specific values in multi-return
result, _ := strconv.Atoi("123")
```

Q12

What are variadic functions?

Difficulty: Easy

```go
// Variadic: accepts any number of arguments of the same type
func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}

sum(1, 2, 3)        // 6
sum(1, 2, 3, 4, 5)  // 15

// Spread slice into variadic
nums := []int{1, 2, 3}
sum(nums...)  // unpack slice

// fmt.Println, fmt.Sprintf are variadic
fmt.Println("a", "b", "c")

// Variadic must be last parameter
func join(sep string, parts ...string) string {
    return strings.Join(parts, sep)
}

// Passing to another variadic function
func wrapper(args ...int) int {
    return sum(args...)  // spread
}
```

Q13

What are function types and closures?

Difficulty: Medium

```go
// Functions are first-class values
type HandlerFunc func(http.ResponseWriter, *http.Request)

// Closure: function that captures variables from outer scope
func makeCounter() func() int {
    count := 0
    return func() int {
        count++
        return count
    }
}

c1 := makeCounter()
c2 := makeCounter()
fmt.Println(c1(), c1(), c1())  // 1 2 3
fmt.Println(c2())              // 1 (independent)

// Closure captures by REFERENCE (not value)
x := 10
f := func() { fmt.Println(x) }
x = 20
f()  // prints 20, not 10!

// Functional options pattern (uses closures)
type Server struct{ port int; timeout time.Duration }
type Option func(*Server)

func WithPort(p int) Option     { return func(s *Server) { s.port = p } }
func WithTimeout(t time.Duration) Option { return func(s *Server) { s.timeout = t } }

func NewServer(opts ...Option) *Server {
    s := &Server{port: 8080, timeout: 30 * time.Second}
    for _, o := range opts { o(s) }
    return s
}
srv := NewServer(WithPort(9090), WithTimeout(time.Minute))
```

Q14

What is panic and recover?

Difficulty: Medium

```go
// panic: runtime error, unwinds stack, runs defers, terminates program
func divide(a, b int) int {
    if b == 0 {
        panic("division by zero")
    }
    return a / b
}

// recover: catches panic, must be called in deferred function
func safeDiv(a, b int) (result int, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("caught panic: %v", r)
        }
    }()
    result = divide(a, b)
    return
}

// Use panic/recover for:
// - Unrecoverable programmer errors (index out of bounds, nil dereference)
// - NOT for normal error flow (use error returns instead)

// HTTP middleware: catch panics to avoid server crash
func recoverMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if r := recover(); r != nil {
                log.Printf("panic: %v\n%s", r, debug.Stack())
                http.Error(w, "internal server error", 500)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

Q15

What are init functions?

Difficulty: Easy

```go
// init(): runs automatically before main(), after package-level vars initialized
// Multiple init() functions allowed per file/package
// Called in order of file import dependency

package db

var pool *sql.DB

func init() {
    var err error
    pool, err = sql.Open("postgres", os.Getenv("DSN"))
    if err != nil {
        log.Fatal(err)
    }
}

// Execution order:
// 1. Package-level variables (in order of declaration)
// 2. init() functions (in order of source files, then declaration)
// 3. main()

// Common uses:
// - Register drivers (database/sql drivers)
// - Set up package-level state
// - Validate config

// Side-effect imports (triggers init() only)
import _ "github.com/lib/pq"        // registers postgres driver
import _ "net/http/pprof"           // registers /debug/pprof handlers
```

Q16

What is the make vs new function?

Difficulty: Medium

```go
// new(T): allocates memory, returns *T (zeroed)
p := new(int)     // *int pointing to 0
u := new(User)    // *User with all fields zeroed

// make(T, args): creates and initializes slices, maps, channels
// Returns the type itself (not a pointer)

// Slice: make([]T, length, capacity)
s := make([]int, 5)      // len=5, cap=5
s := make([]int, 3, 10)  // len=3, cap=10

// Map: make(map[K]V)
m := make(map[string]int)
m := make(map[string]int, 100) // hint initial capacity

// Channel: make(chan T, bufferSize)
ch := make(chan int)     // unbuffered
ch := make(chan int, 10) // buffered, capacity 10

// new vs make:
// new: just allocates, returns pointer
// make: allocates + initializes internal data structures

// You can also use literals:
s := []int{}           // same as make([]int, 0)
m := map[string]int{}  // same as make(map[string]int)
```

Q17

How do strings work in Go?

Difficulty: Medium

```go
// String: immutable sequence of bytes (UTF-8)
s := "Hello, ■"
len(s)      // 10 bytes (■ = 3 bytes in UTF-8)
len([]rune(s)) // 8 characters

// Byte vs rune iteration
for i := 0; i < len(s); i++ {
    fmt.Printf("%x ", s[i])  // byte values
}

for i, r := range s {        // correct Unicode iteration
    fmt.Printf("%d: %c\n", i, r)
}

// String concatenation: + creates new string (immutable)
// For many concatenations use strings.Builder
var b strings.Builder
for i := 0; i < 1000; i++ {
    b.WriteString("hello")
}
result := b.String()

// Common operations
strings.Contains(s, "sub")
strings.HasPrefix(s, "pre")
strings.HasSuffix(s, "suf")
strings.ToLower(s)
strings.TrimSpace(s)
strings.Split(s, ",")
strings.Join(parts, ", ")
strings.ReplaceAll(s, "old", "new")
fmt.Sprintf("formatted %s %d", name, age)

// String ↔ byte slice conversion (copies)
b := []byte(s)
s2 := string(b)

// String ↔ rune slice
r := []rune(s)
s3 := string(r)
```

Q18

What are type conversions in Go?

Difficulty: Easy

```go
// Go requires explicit type conversions (no implicit)
var i int = 42
var f float64 = float64(i)  // explicit
var u uint = uint(f)

// String conversions
s := string(65)          // "A" (from rune/byte)
s := strconv.Itoa(42)    // "42" (int to string)
n, err := strconv.Atoi("42")  // string to int
f, err := strconv.ParseFloat("3.14", 64)
b, err := strconv.ParseBool("true")

// Type assertion (interface to concrete)
var i interface{} = "hello"
s, ok := i.(string)  // safe: ok=true, s="hello"
s := i.(string)      // panics if not string

// Type switch
switch v := i.(type) {
case string:
    fmt.Printf("string: %s\n", v)
case int:
    fmt.Printf("int: %d\n", v)
default:
    fmt.Printf("unknown: %T\n", v)
}

// Unsafe conversions (use with caution)
import "unsafe"
p := unsafe.Pointer(&x)  // any pointer to unsafe.Pointer
n := *(*int64)(p)         // reinterpret bytes
```

Q19

What are Go's built-in functions?

Difficulty: Easy

```go
// Length and capacity
len(s)   // string bytes, slice elements, map pairs, channel buffer used
cap(s)   // slice capacity, channel buffer capacity

// Memory allocation
make(T, ...)  // slices, maps, channels
new(T)        // any type, returns *T

// Append and copy
s = append(s, elem)      // append to slice
s = append(s, other...)  // append slice to slice
n := copy(dst, src)      // copy min(len(dst), len(src)) elements

// Delete map entry
delete(m, key)

// Close channel
close(ch)

// Panic and recover
panic("message")
recover() // only in deferred function

// Print (avoid in prod, use fmt/log)
print("debug")
println("debug")

// Complex numbers
c := complex(real, imag)
r := real(c)
i := imag(c)

// Type conversion-like builtins
string(r)     // rune to string
[]byte(s)     // string to bytes
[]rune(s)     // string to runes
```

Q20

What is the init execution order?

Difficulty: Medium

```go
// Package initialization order:
// 1. All imported packages initialized first (recursive)
// 2. Package-level variables in declaration order
// 3. init() functions in source file order

// Example:
package main

import "fmt"

var x = compute()  // runs before init()

func compute() int {
    fmt.Println("1. package var")
    return 42
}

func init() {
    fmt.Println("2. init()")
}

func main() {
    fmt.Println("3. main()")
}

// Output:
// 1. package var
// 2. init()
// 3. main()

// Multiple init() in same file: run in order
// Multiple init() in same package: run in file order (alphabetical)
// init() cannot be called directly (compiler error)
// init() can reference package-level variables
```

Q21

What is a named return value?

Difficulty: Medium

```go
// Named return values: declared in function signature
func divide(a, b float64) (result float64, err error) {
    if b == 0 {
        err = errors.New("division by zero")
        return  // "naked return": returns named values
    }
    result = a / b
    return  // returns result and nil err
}

// Named returns with defer (powerful pattern)
func loadConfig(path string) (cfg *Config, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("panic loading config: %v", r)
            cfg = nil
        }
    }()
    // parse config...
    return cfg, nil
}

// Downside: naked returns reduce readability in long functions
// Best for: short functions, defer-modify pattern

// Documentation: named returns document what values mean
func httpStatus() (code int, message string) {
    return 200, "OK"
}
```

Q22

How do goroutine stack and heap work?

Difficulty: Hard

```go
// Stack: goroutine-local, grows dynamically (2KB → up to 1GB)
// Heap: shared, managed by GC

// Escape analysis: compiler decides stack vs heap
// go build -gcflags="-m" to see decisions

func stackAllocated() {
    x := 42      // stays on stack (doesn't escape)
    _ = x
}

func heapAllocated() *int {
    x := 42
    return &x    // escapes to heap (returned pointer outlives function)
}

func heapAllocated2() {
    s := make([]int, 0, 1000000)  // large: may go to heap
    _ = s
}

// Minimize heap allocations for hot paths
// Profile with: go tool pprof mem.prof
// Check: runtime.ReadMemStats

var stats runtime.MemStats
runtime.ReadMemStats(&stats)
fmt.Printf("Alloc=%v HeapAlloc=%v\n", stats.Alloc, stats.HeapAlloc)
```

Q23

What are Go's comparison operators and equality?

Difficulty: Easy

```go
// Comparable types: basic types, pointers, arrays (of comparable), structs (of comparable fields)
// NOT comparable: slices, maps, functions

// Struct equality: field-by-field
type Point struct{ X, Y int }
p1 := Point{1, 2}
p2 := Point{1, 2}
fmt.Println(p1 == p2)  // true

// Array equality (unlike slices)
a1 := [3]int{1, 2, 3}
a2 := [3]int{1, 2, 3}
fmt.Println(a1 == a2)  // true

// Slice: use reflect.DeepEqual or manual comparison
s1 := []int{1, 2, 3}
s2 := []int{1, 2, 3}
fmt.Println(reflect.DeepEqual(s1, s2))  // true

// Interface equality: compares (type, value) pairs
var i1 interface{} = 42
var i2 interface{} = 42
fmt.Println(i1 == i2)  // true

// Map key must be comparable
m := map[[2]int]string{{1,2}: "point"}  // [2]int array is comparable
// m := map[[]int]string{}  // COMPILE ERROR: slice not comparable
```

Q24

What are go build tags?

Difficulty: Medium

```go
// Build tags: conditional compilation
// File: server_linux.go
//go:build linux

package main

func platformInfo() string { return "Linux" }

// File: server_windows.go
//go:build windows

package main

func platformInfo() string { return "Windows" }

// Custom tags
//go:build integration

package test

func TestIntegration(t *testing.T) { ... }
// Run: go test -tags integration ./...

// Multiple conditions
//go:build (linux || darwin) && amd64

// Negation
//go:build !windows

// Old style (still supported)
// +build linux,amd64

// Common use: OS-specific code, test categories, feature flags
// File naming convention also works:
// file_linux.go, file_darwin.go, file_amd64.go
```

Q25

What is go generate and go:generate?

Difficulty: Medium

```go
// go:generate: directive that triggers code generation
// Run: go generate ./...

//go:generate stringer -type=Direction
type Direction int
const (
    North Direction = iota
    East; South; West
)

//go:generate mockgen -source=interfaces.go -destination=mocks/mock_service.go

//go:generate protoc --go_out=. proto/service.proto

// Common generators:
// stringer: generates String() method for iota enums
// mockgen: generates mock implementations
// protoc: generates gRPC code from .proto files
// sqlc: generates type-safe DB code from SQL
// wire: dependency injection code generation

// The directive is a comment: //go:generate command args
// Only runs when explicitly invoked: go generate
// Does NOT run during go build or go test automatically
```

Q26

What is the difference between make([]T, 0) and var s []T?

Difficulty: Medium

```go
var s []int          // nil slice: s == nil is true, len=0, cap=0
s := make([]int, 0) // non-nil empty slice: s == nil is false, len=0, cap=0
s := []int{}        // non-nil empty slice (same as make with 0)

// Practical difference: JSON marshaling
var s []int
json.Marshal(s)  // → "null"

s := make([]int, 0)
json.Marshal(s)  // → "[]"

// Both work for append:
var s []int
s = append(s, 1, 2, 3)  // fine

// Prefer var for zero-value semantics
// Prefer make([]T, 0) when returning empty (not null) JSON arrays
```

Q27

What is method set and interface satisfaction?

Difficulty: Hard

```go
// Method set of T: methods declared with value receiver T
// Method set of *T: methods declared with T AND *T receivers

type Writer interface{ Write([]byte) (int, error) }

type Buffer struct{ data []byte }

// Value receiver: in method set of both Buffer and *Buffer
func (b Buffer) String() string { return string(b.data) }

// Pointer receiver: in method set of *Buffer ONLY
func (b *Buffer) Write(p []byte) (int, error) {
    b.data = append(b.data, p...)
    return len(p), nil
}

var w Writer
w = &Buffer{}   // OK: *Buffer has Write (pointer receiver)
w = Buffer{}    // COMPILE ERROR: Buffer does not have Write

// Why? Taking address of interface value is impossible
// Interface holds (type, value); value may not be addressable

// Rule: addressable values auto-dereference for pointer receiver methods
b := Buffer{}
b.Write([]byte("hello"))  // Go auto-takes &b → (&b).Write(...)

// Interface value is NOT addressable, so this fails:
var w Writer = Buffer{}  // can't take &w to call Write
```

Q28

What is the zero value and why is it important?

Difficulty: Easy

```go
// Zero values make Go safe by default
var i int           // 0
var f float64       // 0.0
var b bool          // false
var s string        // ""
var p *int          // nil
var sl []int        // nil (len=0, cap=0)
var m map[string]int // nil
var ch chan int      // nil
var fn func()       // nil
var err error       // nil (interface with nil type and value)

// Useful zero values:
// sync.Mutex: usable without initialization
var mu sync.Mutex
mu.Lock()

// bytes.Buffer: usable without initialization
var buf bytes.Buffer
buf.WriteString("hello")

// Design for zero value:
type Config struct {
    Timeout time.Duration // zero = no timeout (0s means unlimited)
    Debug   bool          // zero = false (disabled)
}
cfg := Config{} // sensible defaults without initialization
```

Q29

What are runes and how do you handle Unicode?

Difficulty: Medium

```go
// rune = int32 = Unicode code point
// string = []byte (UTF-8 encoded)

s := "Hello, ■■"

// WRONG: byte iteration (incorrect for multi-byte chars)
for i := 0; i < len(s); i++ {
    fmt.Printf("%c", s[i])  // garbled for ■■
}

// CORRECT: rune iteration
for _, r := range s {
    fmt.Printf("%c", r)  // correct Unicode characters
}

// String length in runes
runeCount := len([]rune(s))
// or: utf8.RuneCountInString(s)
import "unicode/utf8"
utf8.RuneCountInString(s)

// Reverse a string correctly (byte reversal = garbled)
func reverseString(s string) string {
    runes := []rune(s)
    for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
        runes[i], runes[j] = runes[j], runes[i]
    }
    return string(runes)
}

// Check valid UTF-8
utf8.ValidString(s)

// Normalize (NFC/NFD): golang.org/x/text/unicode/norm
```

Q30

What is the difference between == and reflect.DeepEqual?

Difficulty: Medium

```go
// == : compile-time comparable types only
// reflect.DeepEqual: works on any type, recursive comparison

// Slices: == doesn't compile
s1 := []int{1, 2, 3}
// s1 == s2  // COMPILE ERROR

reflect.DeepEqual(s1, []int{1, 2, 3})  // true

// Maps
m1 := map[string]int{"a": 1}
reflect.DeepEqual(m1, map[string]int{"a": 1})  // true

// Structs with unexported fields
type secret struct{ x int }
s := secret{x: 42}
reflect.DeepEqual(s, secret{x: 42})  // true (accesses unexported fields)

// Nil vs empty slice: DeepEqual distinguishes!
reflect.DeepEqual([]int(nil), []int{})   // FALSE
reflect.DeepEqual([]int(nil), []int(nil)) // true

// Performance: reflect.DeepEqual is slow (reflection overhead)
// For tests: use testify/assert for better error messages
// For production: write type-specific equality functions

// Pointer comparison:
p1, p2 := new(int), new(int)
*p1, *p2 = 42, 42
fmt.Println(p1 == p2)  // false (different addresses)
reflect.DeepEqual(p1, p2)  // true (same pointee value)
```

---

2. Interfaces & Type System

Q31

What are interfaces in Go?

Difficulty: Medium

```go
// Interface: set of method signatures
// Implemented implicitly (no "implements" keyword)
type Animal interface {
    Sound() string
    Name() string
}

type Dog struct{ name string }
func (d Dog) Sound() string { return "Woof" }
func (d Dog) Name() string  { return d.name }

// Dog implicitly implements Animal
var a Animal = Dog{name: "Rex"}
fmt.Println(a.Sound())  // Woof

// Interface value: (type, value) pair
var a Animal            // (nil, nil) - zero value
a = Dog{name: "Rex"}    // (Dog, Dog{name:"Rex"})
a = nil                 // (nil, nil) again

// Interface allows polymorphism without inheritance
func makeSound(a Animal) {
    fmt.Println(a.Name(), "says", a.Sound())
}
makeSound(Dog{name: "Rex"})
makeSound(Cat{name: "Whiskers"})
```

Q32

What is the empty interface and any?

Difficulty: Easy

```go
// interface{} (empty interface): satisfied by ALL types
// Go 1.18+: 'any' is an alias for interface{}

func printAny(v interface{}) {
    fmt.Printf("%T: %v\n", v, v)
}
printAny(42)
printAny("hello")
printAny([]int{1, 2, 3})

// any keyword (Go 1.18+)
func printAny(v any) { ... }

// Type assertion to get back concrete type
func process(v any) {
    switch x := v.(type) {
    case int:    fmt.Printf("int: %d\n", x)
    case string: fmt.Printf("string: %s\n", x)
    case []int:  fmt.Printf("[]int len=%d\n", len(x))
    default:     fmt.Printf("unknown: %T\n", x)
    }
}

// Used in: fmt package, encoding/json, generics alternatives
// Avoid overuse: lose type safety → use generics instead (Go 1.18+)
```

Q33

What are common standard interfaces?

Difficulty: Medium

```go
// io.Reader: reads bytes
type Reader interface {
    Read(p []byte) (n int, err error)
}

// io.Writer: writes bytes
type Writer interface {
    Write(p []byte) (n int, err error)
}

// io.Closer
type Closer interface { Close() error }

// io.ReadWriter, io.ReadWriteCloser: composed interfaces
type ReadWriter interface { Reader; Writer }

// fmt.Stringer: String() for custom fmt output
type Stringer interface { String() string }

// error interface
type error interface { Error() string }

// sort.Interface: sort.Sort() uses this
type Interface interface {
    Len() int
    Less(i, j int) bool
    Swap(i, j int)
}

// http.Handler: HTTP request handler
type Handler interface {
    ServeHTTP(ResponseWriter, *Request)
}

// Implementing multiple interfaces
type File struct{ *os.File }
// os.File implements io.Reader, io.Writer, io.Closer, io.Seeker
var r io.Reader = &File{}
var w io.Writer = &File{}
var c io.Closer = &File{}
```

Q34

What is interface embedding?

Difficulty: Medium

```go
// Embed interfaces to compose larger interfaces
type Reader interface {
    Read(p []byte) (n int, err error)
}
type Writer interface {
    Write(p []byte) (n int, err error)
}
type ReadWriter interface {
    Reader  // embed
    Writer  // embed
}

// Custom composed interface
type Store interface {
    io.Reader    // read data
    io.Writer    // write data
    io.Closer    // close
    Flush() error // custom method
}

// Value satisfying composed interface must implement ALL methods
type MyStore struct{}
func (s *MyStore) Read(p []byte) (int, error) { ... }
func (s *MyStore) Write(p []byte) (int, error) { ... }
func (s *MyStore) Close() error { ... }
func (s *MyStore) Flush() error { ... }

var s Store = &MyStore{}  // OK: implements all 4 methods

// Narrowing: pass larger interface where smaller expected
var rw io.ReadWriter = &MyStore{}
var r io.Reader = rw  // ok: ReadWriter satisfies Reader
```

Q35

What is an interface nil trap?

Difficulty: Hard

```go
// TRAP: interface holds (type, value) — both must be nil for interface == nil

type MyError struct{ msg string }
func (e *MyError) Error() string { return e.msg }

func mayFail() error {
    var err *MyError = nil  // typed nil pointer
    // ... some logic
    return err  // RETURNS NON-NIL INTERFACE! (type=*MyError, value=nil)
}

err := mayFail()
if err != nil {  // TRUE! err is non-nil (has type info)
    fmt.Println("error:", err)  // prints: error: <nil>
}

// Fix: return untyped nil
func mayFail() error {
    var err *MyError
    if someCondition {
        err = &MyError{msg: "failed"}
    }
    if err != nil {
        return err  // ok: returning concrete non-nil error
    }
    return nil  // returns (nil, nil) interface
}

// Or use typed return
func mayFail() (result string, err error) {
    // only assign err if actually errored
    return "ok", nil
}
```

Q36

What are generics in Go (1.18+)?

Difficulty: Hard

```go
// Type parameters: [T constraint]
func Map[T, U any](s []T, f func(T) U) []U {
    result := make([]U, len(s))
    for i, v := range s {
        result[i] = f(v)
    }
    return result
}

doubled := Map([]int{1,2,3}, func(x int) int { return x * 2 })
lengths  := Map([]string{"hi","hello"}, func(s string) int { return len(s) })

// Constraints
type Number interface {
    ~int | ~int32 | ~int64 | ~float32 | ~float64
}

func Sum[T Number](nums []T) T {
    var total T
    for _, n := range nums {
        total += n
    }
    return total
}

// Generic struct
type Stack[T any] struct {
    items []T
}
func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }
func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 { return zero, false }
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}

// comparable constraint for map keys
func Keys[K comparable, V any](m map[K]V) []K {
    keys := make([]K, 0, len(m))
    for k := range m { keys = append(keys, k) }
    return keys
}
```

Q37

What is the difference between concrete type and interface type storage?

Difficulty: Hard

```go
// Interface value has two pointers: *itab (type info + method table) + *data
// Small values stored directly; large values heap-allocated

// Cost of interface:
// 1. Indirect method dispatch (vtable call) vs direct call
// 2. Heap allocation when value escapes
// 3. Cannot inline interface method calls

// Benchmark: interface dispatch overhead
type Adder interface{ Add(int, int) int }
type ConcreteAdder struct{}
func (c ConcreteAdder) Add(a, b int) int { return a + b }

// Direct call: ~0.3 ns/op
// Interface call: ~1-2 ns/op (indirect dispatch)

// When interface overhead matters:
// - Tight inner loops processing millions of items
// - Fix: use generics or type-specific code

// When it doesn't matter:
// - I/O bound operations
// - DB/network calls
// - Most business logic

// reflect.TypeOf and reflect.ValueOf work with interfaces
var i interface{} = 42
fmt.Println(reflect.TypeOf(i))  // int
fmt.Println(reflect.ValueOf(i)) // 42
```

Q38

How do you check if a type implements an interface at compile time?

Difficulty: Easy

```go
// Compile-time interface satisfaction check
// If *MyService doesn't implement Service → compile error
var _ Service = (*MyService)(nil)
var _ io.Reader = (*MyReader)(nil)
var _ http.Handler = (*MyHandler)(nil)

// Using it in practice
type Service interface {
    GetUser(ctx context.Context, id int64) (*User, error)
    CreateUser(ctx context.Context, u *User) error
}

type userService struct{ repo UserRepository }

// This line causes compile error if userService doesn't implement Service
var _ Service = (*userService)(nil)

// Pattern: place near interface definition or implementation
// Purpose: catch missing method implementations early
```

Q39

What is duck typing in Go?

Difficulty: Easy

```go
// Duck typing: "if it quacks like a duck, it's a duck"
// Go uses structural typing: any type with required methods satisfies interface

// You don't need to declare "implements":
type Quacker interface{ Quack() }

type Duck struct{}
func (d Duck) Quack() { fmt.Println("Quack!") }

type Person struct{ Name string }
func (p Person) Quack() { fmt.Println(p.Name, "says: quack!") }

func makeItQuack(q Quacker) { q.Quack() }

makeItQuack(Duck{})
makeItQuack(Person{Name: "Alice"})
// No inheritance, no "implements Duck" declaration needed

// Power: external types can satisfy your interfaces
// os.File satisfies io.Reader, io.Writer, io.Closer — you defined your interface, it works!

// Adapter pattern using duck typing
type LegacyLogger struct{}
func (l *LegacyLogger) Log(msg string) { ... }

// Adapter to satisfy new interface
type LogAdapter struct{ legacy *LegacyLogger }
func (a *LogAdapter) Write(p []byte) (int, error) {
    a.legacy.Log(string(p))
    return len(p), nil
}
var w io.Writer = &LogAdapter{legacy: &LegacyLogger{}}
```

Q40

What are type aliases vs type definitions?

Difficulty: Medium

```go
// Type definition: creates NEW type (not interchangeable)
type Celsius float64
type Fahrenheit float64

c := Celsius(100)
f := Fahrenheit(212)
// c == f  // COMPILE ERROR: different types
// c + f   // COMPILE ERROR

// Type definition adds methods
func (c Celsius) ToFahrenheit() Fahrenheit {
    return Fahrenheit(c*9/5 + 32)
}

// Type alias: just another name (fully interchangeable)
type MyInt = int  // alias, not new type
var x MyInt = 42
var y int = x   // OK: same underlying type

// Aliases used for:
// Gradual code migration (move type between packages)
// Convenience alias (io/ioutil → io)

// Underlying type access:
// Can convert between type and its underlying type
var c Celsius = 100
var f float64 = float64(c)  // explicit conversion

// iota with type definitions
type Color int
const (
    Red Color = iota
    Green
    Blue
)
```

Q41-Q50: More Interfaces & Types

Q41

What is the Stringer interface and how to implement it?

```go
type Order struct{ ID int64; Amount float64; Status string }

func (o Order) String() string {
    return fmt.Sprintf("Order{id=%d amount=%.2f status=%s}", o.ID, o.Amount, o.Status)
}

fmt.Println(order)        // calls String() automatically
fmt.Printf("%v\n", order) // calls String()
fmt.Printf("%+v\n", order) // struct field names (ignores Stringer)
fmt.Printf("%#v\n", order) // Go syntax (ignores Stringer)
```

Q42

What is the error interface and custom errors?

```go
// Built-in error interface
type error interface { Error() string }

// Custom error type (with context)
type ValidationError struct {
    Field   string
    Message string
}
func (e *ValidationError) Error() string {
    return fmt.Sprintf("validation error: field=%s msg=%s", e.Field, e.Message)
}

// Sentinel errors (for comparison)
var ErrNotFound = errors.New("not found")
var ErrUnauthorized = errors.New("unauthorized")

// Wrapped errors (Go 1.13+)
err := fmt.Errorf("get user: %w", ErrNotFound)
errors.Is(err, ErrNotFound)  // true (unwraps chain)

var ve *ValidationError
errors.As(err, &ve)  // true if err chain contains *ValidationError
```

Q43

What are type assertions vs type switches for performance?

```go
// Type assertion: single type check
if s, ok := v.(string); ok { use(s) }

// Type switch: multiple types (compiled to jump table for small N)
switch v := i.(type) {
case string:  handleString(v)
case int:     handleInt(v)
case []byte:  handleBytes(v)
}

// Performance: type switch ≈ series of if-else
// For 2-3 types: similar performance
// For many types: type switch may use hash/jump table

// Interface + generics comparison:
// Interfaces: runtime dispatch, flexible
// Generics: compile-time specialization, faster
```

Q44

What is reflect package and when to use it?

```go
import "reflect"

// Get type info
t := reflect.TypeOf(x)
fmt.Println(t.Name(), t.Kind())

// Get/set value
v := reflect.ValueOf(&x).Elem()
v.SetInt(100)

// Iterate struct fields
t := reflect.TypeOf(User{})
for i := 0; i < t.NumField(); i++ {
    field := t.Field(i)
    fmt.Printf("%s %s tag=%s\n", field.Name, field.Type, field.Tag.Get("json"))
}

// Use reflect for:
// - JSON/YAML/XML marshaling (encoding packages use it)
// - ORM/database mapping
// - Dependency injection frameworks
// - Template engines

// Avoid reflect when:
// - Performance critical (10-100x slower than direct code)
// - Generics can solve the problem (Go 1.18+)
```

Q45

What is interface composition for testability?

```go
// Small interfaces are easier to mock and test
type UserGetter interface { GetUser(ctx context.Context, id int64) (*User, error) }
type UserCreator interface { CreateUser(ctx context.Context, u *User) error }
type UserStore interface {
    UserGetter
    UserCreator
}

// Service only needs what it uses (Interface Segregation)
type EmailService struct { getter UserGetter }  // only needs GetUser
func (s *EmailService) SendWelcome(ctx context.Context, userID int64) error {
    user, err := s.getter.GetUser(ctx, userID)
    if err != nil { return err }
    return sendEmail(user.Email)
}

// In tests: mock only GetUser (not entire UserStore)
type mockGetter struct{}
func (m mockGetter) GetUser(_ context.Context, id int64) (*User, error) {
    return &User{ID: id, Email: "test@test.com"}, nil
}
svc := &EmailService{getter: mockGetter{}}
```

Q46

What is the io.Reader composition pattern?

```go
// Chain readers using composition (decorator pattern)
file, _ := os.Open("data.gz")
defer file.Close()

gzReader, _ := gzip.NewReader(file)  // decompress
defer gzReader.Close()

bufReader := bufio.NewReader(gzReader)  // buffered read

scanner := bufio.NewScanner(bufReader)  // line scanner
for scanner.Scan() {
    line := scanner.Text()
    process(line)
}

// All implement io.Reader — composable pipeline
// No data loaded into memory all at once (streaming)
```

Q47

What are typed nil interfaces?

```go
// Typed nil: non-nil interface containing nil pointer
var p *MyError = nil
var err error = p
fmt.Println(err == nil)  // FALSE! err has type *MyError

// This causes subtle bugs:
func getError(fail bool) error {
    var err *MyError
    if fail { err = &MyError{"failed"} }
    return err  // always returns non-nil interface!
}

// Fix: only return err if non-nil
func getError(fail bool) error {
    if fail { return &MyError{"failed"} }
    return nil  // untyped nil
}
```

Q48

What is the comparable constraint in generics?

```go
// comparable: types that support == and != operators
// Used in generic functions needing equality comparison

func Contains[T comparable](s []T, v T) bool {
    for _, item := range s {
        if item == v { return true }
    }
    return false
}

Contains([]int{1,2,3}, 2)           // true
Contains([]string{"a","b"}, "c")    // false

// Maps require comparable keys
func MapKeys[K comparable, V any](m map[K]V) []K {
    keys := make([]K, 0, len(m))
    for k := range m { keys = append(keys, k) }
    return keys
}
```

Q49

What is embedding interfaces in structs?

```go
// Embed interface in struct: partial implementation / override pattern
type Logger interface {
    Log(msg string)
    Warn(msg string)
    Error(msg string)
}

// Only override specific methods
type TestLogger struct {
    Logger          // embed interface (delegates all to embedded)
    logged []string // capture logs
}

func (t *TestLogger) Log(msg string) {
    t.logged = append(t.logged, msg)  // override Log
    // Warn and Error still delegate to embedded Logger
}
```

Q50

What is the fmt package interface for custom formatting?

```go
// fmt.Stringer: String() string
// fmt.GoStringer: GoString() string (for %#v)
// fmt.Formatter: Format(f fmt.State, verb rune) (custom verbs)

type Money struct{ Amount int64; Currency string }

func (m Money) String() string {
    return fmt.Sprintf("%s %.2f", m.Currency, float64(m.Amount)/100)
}

func (m Money) GoString() string {
    return fmt.Sprintf("Money{Amount: %d, Currency: %q}", m.Amount, m.Currency)
}

m := Money{Amount: 1099, Currency: "USD"}
fmt.Println(m)      // USD 10.99
fmt.Printf("%v", m) // USD 10.99
fmt.Printf("%#v", m) // Money{Amount: 1099, Currency: "USD"}
```

---

3. Concurrency — Goroutines & Channels

Q51

What are goroutines and how do they differ from threads?

Difficulty: Medium

```go
// Goroutine: lightweight, managed by Go runtime (not OS thread)
// Thread: OS-managed, ~1MB stack, expensive context switch
// Goroutine: ~2KB stack (grows dynamically), cheap context switch

// Launch goroutine
go func() {
    fmt.Println("running in goroutine")
}()

go processOrder(order)

// Goroutines are multiplexed onto OS threads by Go scheduler (GMP model)
// M goroutines run on N OS threads (M >> N possible)

// Start 10,000 goroutines (impossible with threads)
for i := 0; i < 10000; i++ {
    go func(id int) {
        time.Sleep(time.Second)
        fmt.Println("goroutine", id, "done")
    }(i)
}

// Goroutine lifecycle: exits when its function returns
// Goroutine leak: goroutine stuck forever (waiting on channel, timer, etc.)
// Always ensure goroutines can exit (use context.Context for cancellation)
```

Q52

What are channels and how do they work?

Difficulty: Medium

```go
// Channel: goroutine-safe communication pipe
ch := make(chan int)       // unbuffered
ch := make(chan int, 10)   // buffered, capacity 10

// Send (blocks until receiver ready for unbuffered)
ch <- 42

// Receive (blocks until sender ready for unbuffered)
v := <-ch

// Check if channel closed
v, ok := <-ch
if !ok { fmt.Println("channel closed") }

// Close channel
close(ch)
// After close: receives return zero value + ok=false
// Sending to closed channel: PANIC

// Range over channel: receives until closed
for v := range ch {
    fmt.Println(v)
}

// Buffered channel:
// Send: blocks only when buffer full
// Receive: blocks only when buffer empty
ch := make(chan int, 3)
ch <- 1   // no block
ch <- 2   // no block
ch <- 3   // no block
ch <- 4   // BLOCKS: buffer full
```

Q53

What are the channel directions (chan<-, <-chan)?

Difficulty: Medium

```go
// Bidirectional channel: chan T
// Send-only channel:     chan<- T
// Receive-only channel:  <-chan T

// Channel direction in function signatures for safety
func producer(out chan<- int) {    // can only SEND
    for i := 0; i < 5; i++ {
        out <- i
    }
    close(out)
}

func consumer(in <-chan int) {     // can only RECEIVE
    for v := range in {
        fmt.Println(v)
    }
}

func main() {
    ch := make(chan int, 5)  // bidirectional
    go producer(ch)          // bidirectional auto-converts to chan<-
    consumer(ch)             // bidirectional auto-converts to <-chan
}

// Direction enforcement prevents:
// - Consumer accidentally sending to input channel
// - Producer accidentally receiving from output channel
// Compile-time safety: sending to <-chan = compile error
```

Q54

What is select statement?

Difficulty: Medium

```go
// select: wait on multiple channel operations
select {
case v := <-ch1:
    fmt.Println("received from ch1:", v)
case v := <-ch2:
    fmt.Println("received from ch2:", v)
case ch3 <- 42:
    fmt.Println("sent to ch3")
default:
    fmt.Println("no channels ready (non-blocking)")
}

// Without default: blocks until one case is ready
// With default: non-blocking (poll)

// Timeout pattern
select {
case result := <-computeAsync():
    fmt.Println("result:", result)
case <-time.After(5 * time.Second):
    fmt.Println("timeout!")
}

// Cancellation with context
func worker(ctx context.Context, ch <-chan Job) {
    for {
        select {
        case job, ok := <-ch:
            if !ok { return }  // channel closed
            process(job)
        case <-ctx.Done():
            return  // cancelled
        }
    }
}

// Random selection: if multiple cases ready, Go picks one uniformly at random
// Use: fairness in priority queues
```

Q55

What is the done channel pattern?

Difficulty: Medium

```go
// Done channel: signal goroutines to stop
done := make(chan struct{})  // struct{} = zero bytes

// Worker goroutine
go func() {
    for {
        select {
        case <-done:
            return  // received signal to stop
        case job := <-jobCh:
            processJob(job)
        }
    }
}()

// Stop the goroutine
close(done)  // closing broadcasts to ALL receivers

// Modern approach: context.Context (preferred)
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

go func() {
    for {
        select {
        case <-ctx.Done():
            return
        case job := <-jobCh:
            processJob(job)
        }
    }
}()

cancel()  // stops the goroutine
```

Q56

What is context.Context and how do you use it?

Difficulty: Hard

```go
import "context"

// Context: carries deadlines, cancellation signals, request-scoped values
// Pass as first argument to every function that does I/O

// Background and TODO (root contexts)
ctx := context.Background()  // empty context, never cancelled
ctx := context.TODO()        // placeholder when unsure

// WithCancel: manual cancellation
ctx, cancel := context.WithCancel(parent)
defer cancel()  // always call cancel to release resources

// WithTimeout: auto-cancel after duration
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
defer cancel()

// WithDeadline: auto-cancel at specific time
ctx, cancel := context.WithDeadline(parent, time.Now().Add(5*time.Second))
defer cancel()

// WithValue: attach request-scoped data (use sparingly)
type ctxKey string
ctx = context.WithValue(ctx, ctxKey("userID"), int64(42))
userID := ctx.Value(ctxKey("userID")).(int64)

// Check cancellation
select {
case <-ctx.Done():
    return ctx.Err()  // context.Canceled or context.DeadlineExceeded
default:
}

// HTTP request context (already has deadline)
func handler(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()
    result, err := db.QueryContext(ctx, "SELECT ...")
    // Query automatically cancelled if client disconnects!
}
```

Q57

What is a goroutine leak and how do you prevent it?

Difficulty: Hard

```go
// Goroutine leak: goroutine stuck waiting, never exits
// Common causes:

// 1. Channel send/receive with no goroutine on other side
go func() {
    ch <- result  // LEAK: if nobody reads ch
}()

// Fix: buffered channel or ensure receiver exists
ch := make(chan int, 1)  // buffered: send won't block
go func() { ch <- result }()

// 2. No cancellation support
go func() {
    for {
        processNext()  // LEAK: infinite loop, no way to stop
    }
}()

// Fix: context-based cancellation
go func(ctx context.Context) {
    for {
        select {
        case <-ctx.Done(): return
        default: processNext()
        }
    }
}(ctx)

// 3. Waiting on response from crashed sender
ch := make(chan Response)
go func() {
    resp, err := callExternalService()
    if err != nil { return }  // LEAK: never sends, receiver blocked
    ch <- resp
}()
result := <-ch  // blocks forever if goroutine returned early

// Fix: always send (even on error) or use select with ctx
go func() {
    resp, err := callExternalService()
    if err != nil { ch <- Response{Err: err}; return }
    ch <- Response{Data: resp}
}()

// Detect leaks: goleak library in tests
import "go.uber.org/goleak"
func TestSomething(t *testing.T) {
    defer goleak.VerifyNone(t)
    // test code
}
```

Q58

What is the pipeline pattern with channels?

Difficulty: Hard

```go
// Pipeline: chain of stages connected by channels
// Each stage: receive from upstream, process, send downstream

// Stage 1: generate numbers
func generate(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums {
            out <- n
        }
    }()
    return out
}

// Stage 2: square numbers
func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in {
            out <- n * n
        }
    }()
    return out
}

// Stage 3: filter
func filter(in <-chan int, pred func(int) bool) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in {
            if pred(n) { out <- n }
        }
    }()
    return out
}

// Compose pipeline
for result := range filter(square(generate(1,2,3,4,5)), func(n int) bool { return n > 5 }) {
    fmt.Println(result)  // 9, 16, 25
}
```

Q59

What is fan-out and fan-in with goroutines?

Difficulty: Hard

```go
// Fan-out: distribute work across N goroutines
func fanOut(in <-chan Job, n int) []<-chan Result {
    channels := make([]<-chan Result, n)
    for i := 0; i < n; i++ {
        channels[i] = worker(in)  // each worker reads from same input channel
    }
    return channels
}

// Fan-in: merge N channels into one
func fanIn(channels ...<-chan Result) <-chan Result {
    merged := make(chan Result)
    var wg sync.WaitGroup

    output := func(c <-chan Result) {
        defer wg.Done()
        for r := range c { merged <- r }
    }

    wg.Add(len(channels))
    for _, c := range channels { go output(c) }

    go func() {
        wg.Wait()
        close(merged)
    }()
    return merged
}

// Usage: process jobs in parallel, collect results
jobs := generate(jobList...)
workers := fanOut(jobs, runtime.NumCPU())
results := fanIn(workers...)
for r := range results {
    collect(r)
}
```

Q60

What is the worker pool pattern?

Difficulty: Hard

```go
type WorkerPool struct {
    jobs    chan Job
    results chan Result
    wg      sync.WaitGroup
}

func NewWorkerPool(numWorkers int) *WorkerPool {
    p := &WorkerPool{
        jobs:    make(chan Job, numWorkers*2),
        results: make(chan Result, numWorkers*2),
    }

    for i := 0; i < numWorkers; i++ {
        p.wg.Add(1)
        go p.worker()
    }

    // Close results when all workers done
    go func() {
        p.wg.Wait()
        close(p.results)
    }()

    return p
}

func (p *WorkerPool) worker() {
    defer p.wg.Done()
    for job := range p.jobs {
        result := processJob(job)
        p.results <- result
    }
}

func (p *WorkerPool) Submit(job Job) { p.jobs <- job }
func (p *WorkerPool) Close()         { close(p.jobs) }
func (p *WorkerPool) Results() <-chan Result { return p.results }

// Usage
pool := NewWorkerPool(runtime.NumCPU())
for _, job := range jobs { pool.Submit(job) }
pool.Close()
for r := range pool.Results() { handle(r) }
```

Q61

What is unbuffered vs buffered channel behavior?

Difficulty: Medium

```go
// Unbuffered: synchronous rendezvous
// Send BLOCKS until receiver is ready
// Receive BLOCKS until sender is ready
// Guarantees: receiver has the value BEFORE sender continues

ch := make(chan int)
go func() {
    ch <- 42  // blocks here until main receives
    fmt.Println("sent!")  // only prints AFTER receive
}()
v := <-ch
fmt.Println("received", v)
// Output: received 42, sent! (guaranteed order)

// Buffered: asynchronous (up to buffer size)
// Send BLOCKS only when buffer full
// Receive BLOCKS only when buffer empty
ch := make(chan int, 3)
ch <- 1   // no block
ch <- 2   // no block
ch <- 3   // no block
// ch <- 4 would block

// Buffered channel as semaphore (limit concurrency)
sem := make(chan struct{}, 5)  // max 5 concurrent
for _, url := range urls {
    sem <- struct{}{}  // acquire
    go func(u string) {
        defer func() { <-sem }()  // release
        fetch(u)
    }(url)
}
```

Q62

How do you implement a timeout for goroutines?

Difficulty: Medium

```go
// Pattern 1: time.After
func doWithTimeout(timeout time.Duration) (Result, error) {
    ch := make(chan Result, 1)  // buffered: goroutine won't leak
    go func() {
        ch <- expensiveComputation()
    }()

    select {
    case result := <-ch:
        return result, nil
    case <-time.After(timeout):
        return Result{}, errors.New("timeout")
    }
}

// Pattern 2: context.WithTimeout (preferred)
func doWithContext(ctx context.Context) (Result, error) {
    ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    ch := make(chan Result, 1)
    go func() {
        result := expensiveComputation()
        select {
        case ch <- result:
        case <-ctx.Done():  // don't block if already cancelled
        }
    }()

    select {
    case result := <-ch:
        return result, nil
    case <-ctx.Done():
        return Result{}, ctx.Err()
    }
}

// time.NewTimer (more efficient than time.After for reuse)
timer := time.NewTimer(5 * time.Second)
defer timer.Stop()
select {
case v := <-ch: use(v)
case <-timer.C: timeout()
}
```

Q63

What is channel axioms (what happens with nil/closed channels)?

Difficulty: Hard

```go
// OPERATIONS ON CHANNELS:
//                  nil     open    closed
// Send:            block   works   PANIC
// Receive:         block   works   zero+false
// Close:           PANIC   works   PANIC

var ch chan int  // nil channel

// nil channel:
ch <- 1          // blocks forever
<-ch             // blocks forever
close(ch)        // PANIC

// closed channel:
close(ch)
ch <- 1          // PANIC
v, ok := <-ch   // v=0, ok=false (immediate, no block)
for v := range ch { }  // range exits immediately

// Safe close pattern: only close from sender side
// Never close from receiver side

// Multiple goroutines: use sync.Once to prevent double close
var once sync.Once
closeOnce := func() { once.Do(func() { close(ch) }) }
```

Q64

What is the for-select loop pattern?

Difficulty: Medium

```go
// Most common goroutine pattern: event loop
func eventLoop(ctx context.Context, jobs <-chan Job, results chan<- Result) {
    for {
        select {
        case <-ctx.Done():
            return
        case job, ok := <-jobs:
            if !ok {
                return  // jobs channel closed
            }
            result := process(job)
            select {
            case results <- result:
            case <-ctx.Done():
                return
            }
        }
    }
}

// With ticker (periodic work)
func periodic(ctx context.Context) {
    ticker := time.NewTicker(time.Minute)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            doPeriodicWork()
        }
    }
}
```

Q65

What is the errgroup package?

Difficulty: Hard

```go
import "golang.org/x/sync/errgroup"

// errgroup: run goroutines, collect first error, wait for all
func fetchAll(ctx context.Context, urls []string) ([]string, error) {
    g, ctx := errgroup.WithContext(ctx)
    results := make([]string, len(urls))

    for i, url := range urls {
        i, url := i, url  // capture loop vars
        g.Go(func() error {
            body, err := fetch(ctx, url)
            if err != nil { return err }
            results[i] = body
            return nil
        })
    }

    if err := g.Wait(); err != nil {
        return nil, err  // returns first error
    }
    return results, nil
}

// With limit (semaphore built in)
g := new(errgroup.Group)
g.SetLimit(10)  // max 10 goroutines at once

for _, job := range jobs {
    job := job
    g.Go(func() error {
        return process(job)
    })
}
err := g.Wait()
```

Q66

What is the semaphore pattern?

Difficulty: Medium

```go
// Semaphore: limit concurrent goroutines
// Pattern 1: buffered channel
sem := make(chan struct{}, maxConcurrent)

var wg sync.WaitGroup
for _, item := range items {
    wg.Add(1)
    sem <- struct{}{}  // acquire
    go func(x Item) {
        defer wg.Done()
        defer func() { <-sem }()  // release
        process(x)
    }(item)
}
wg.Wait()

// Pattern 2: golang.org/x/sync/semaphore
import "golang.org/x/sync/semaphore"

sem := semaphore.NewWeighted(10)
for _, item := range items {
    if err := sem.Acquire(ctx, 1); err != nil { break }
    go func(x Item) {
        defer sem.Release(1)
        process(x)
    }(item)
}
sem.Acquire(ctx, 10)  // wait for all to complete
sem.Release(10)
```

Q67

What are common concurrency bugs in Go?

Difficulty: Hard

```go
// Bug 1: Data race (unsynchronized concurrent access)
counter := 0
var wg sync.WaitGroup
for i := 0; i < 1000; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        counter++  // DATA RACE
    }()
}
// Fix: sync.Mutex or atomic
var counter int64
atomic.AddInt64(&counter, 1)

// Bug 2: Goroutine leak (channel never read)
ch := make(chan int)
go func() { ch <- compute() }()  // LEAK if nobody reads ch
// Fix: buffered channel or ensure receiver

// Bug 3: Loop variable capture (pre-Go 1.22)
for i := 0; i < 5; i++ {
    go func() { fmt.Println(i) }()  // all print 5!
}
// Fix: shadow variable
for i := 0; i < 5; i++ {
    i := i
    go func() { fmt.Println(i) }()
}

// Bug 4: Deadlock (goroutines waiting on each other)
ch1, ch2 := make(chan int), make(chan int)
go func() { ch1 <- 1; v := <-ch2; _ = v }()
ch2 <- 1  // DEADLOCK: main sends to ch2, goroutine sends to ch1, nobody receives ch1
v := <-ch1

// Bug 5: Closing closed channel → panic
// Use sync.Once or ownership model (only sender closes)
```

Q68

What is the sync/atomic package?

Difficulty: Medium

```go
import "sync/atomic"

// Atomic operations: lock-free, thread-safe for simple types
var counter int64

// Add and return new value
atomic.AddInt64(&counter, 1)
atomic.AddInt64(&counter, -1)

// Load and Store
val := atomic.LoadInt64(&counter)
atomic.StoreInt64(&counter, 0)

// Compare-and-swap (CAS)
old, new := int64(5), int64(10)
swapped := atomic.CompareAndSwapInt64(&counter, old, new)
// Only sets if current value == old

// Swap: set and return old value
prev := atomic.SwapInt64(&counter, 0)

// atomic.Value for any type (safe read/write)
var v atomic.Value
v.Store(config)          // store any type
cfg := v.Load().(Config) // load and type assert

// Use atomic for:
// Simple counters, flags, pointers
// When mutex overhead is too high
// Lock-free data structures

// Don't use atomic for:
// Complex operations requiring multiple atomic steps
// Protecting large structs (use mutex)
```

Q69

What is the race detector?

Difficulty: Medium

```bash
# Run with race detector
go test -race ./...
go run -race main.go
go build -race -o myapp main.go

# Race detector:
# - Instruments memory accesses at compile time
# - Reports data races at runtime with goroutine stack traces
# - 2-20x slowdown, 5-10x memory increase
# - Use in development and CI, not production

# Example output:
# WARNING: DATA RACE
# Write at 0x... by goroutine 7:
#   main.(*counter).increment()
# Read at 0x... by goroutine 8:
#   main.(*counter).value()

# Always run tests with -race
# Add to CI pipeline:
go test -race -count=1 ./...
```

Q70

What is the difference between concurrency and parallelism?

Difficulty: Easy

```go
Concurrency: dealing with multiple things at once (structure)
  - Multiple goroutines running conceptually simultaneously
  - Interleaved execution on single core
  - About design and composition

Parallelism: doing multiple things at once (execution)
  - Multiple goroutines literally running simultaneously
  - Multiple CPU cores required
  - About performance

Go enables both:
  GOMAXPROCS = 1: concurrent but not parallel
  GOMAXPROCS = 8: concurrent AND parallel

// Set number of OS threads (defaults to num CPU cores)
runtime.GOMAXPROCS(runtime.NumCPU())

// Example:
// Concurrent: web server handles 1000 requests (goroutines)
//   - Most are waiting on I/O (network, DB)
//   - Can run on 1 CPU via context switching
// Parallel: image processing splits image into tiles
//   - Each tile processed simultaneously
//   - Actually uses multiple CPUs
```

---

4. Concurrency — sync Package & Patterns

Q71

What is sync.Mutex and sync.RWMutex?

Difficulty: Medium

```go
// Mutex: mutual exclusion lock
type SafeCounter struct {
    mu    sync.Mutex
    count int
}

func (c *SafeCounter) Increment() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.count++
}

func (c *SafeCounter) Value() int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.count
}

// RWMutex: multiple readers OR one writer
type Cache struct {
    mu   sync.RWMutex
    data map[string]string
}

func (c *Cache) Get(key string) (string, bool) {
    c.mu.RLock()          // multiple goroutines can RLock simultaneously
    defer c.mu.RUnlock()
    v, ok := c.data[key]
    return v, ok
}

func (c *Cache) Set(key, value string) {
    c.mu.Lock()           // exclusive write lock
    defer c.mu.Unlock()
    c.data[key] = value
}

// RWMutex: better than Mutex when reads >> writes
// Mutex: simpler, fine for mostly writes
```

Q72

What is sync.WaitGroup?

Difficulty: Easy

```go
// WaitGroup: wait for a collection of goroutines to finish
var wg sync.WaitGroup

for i := 0; i < 10; i++ {
    wg.Add(1)  // increment BEFORE launching goroutine
    go func(id int) {
        defer wg.Done()  // decrement when goroutine exits
        processItem(id)
    }(i)
}

wg.Wait()  // blocks until counter reaches 0
fmt.Println("all done")

// Common mistakes:
// 1. wg.Add inside goroutine (race condition)
// 2. Forgetting wg.Done on error path (use defer)
// 3. Reusing WaitGroup before Wait returns

// Pattern: process batch with WaitGroup
func processBatch(items []Item) error {
    var wg sync.WaitGroup
    errs := make(chan error, len(items))

    for _, item := range items {
        wg.Add(1)
        go func(i Item) {
            defer wg.Done()
            if err := process(i); err != nil {
                errs <- err
            }
        }(item)
    }

    wg.Wait()
    close(errs)

    for err := range errs {
        if err != nil { return err }
    }
    return nil
}
```

Q73

What is sync.Once?

Difficulty: Easy

```go
// sync.Once: execute function exactly once (goroutine-safe)
var (
    instance *Database
    once     sync.Once
)

func GetDatabase() *Database {
    once.Do(func() {
        instance = &Database{}
        instance.Connect()
    })
    return instance
}

// Thread-safe singleton pattern
// Even if 1000 goroutines call GetDatabase() simultaneously:
// - Connect() runs exactly ONCE
// - All callers get the same instance

// sync.Once for cleanup
type Resource struct {
    closeOnce sync.Once
}

func (r *Resource) Close() error {
    var err error
    r.closeOnce.Do(func() {
        err = r.cleanup()
    })
    return err  // safe to call Close() multiple times
}

// sync.Once resets after Do completes
// Cannot reset sync.Once (by design)
```

Q74

What is sync.Map?

Difficulty: Hard

```go
// sync.Map: concurrent-safe map, no external lock needed
// Optimized for:
//   - Write-once, read-many (cache)
//   - Keys written and read by disjoint goroutines

var m sync.Map

// Store
m.Store("key", "value")
m.Store("count", int64(42))

// Load
val, ok := m.Load("key")
if ok {
    fmt.Println(val.(string))
}

// LoadOrStore: atomic get-or-set
actual, loaded := m.LoadOrStore("key", "default")
// actual = existing value if loaded=true, or "default" if just stored

// Delete
m.Delete("key")

// Range (iterate, cannot modify while ranging)
m.Range(func(k, v interface{}) bool {
    fmt.Printf("%v: %v\n", k, v)
    return true  // return false to stop iteration
})

// sync.Map vs map + RWMutex:
// sync.Map is FASTER for read-heavy with few writes (amortized)
// map + RWMutex is better for write-heavy or range-heavy
// sync.Map has no Len() — if you need count, use map+mutex
```

Q75

What is sync.Pool?

Difficulty: Hard

```go
// sync.Pool: reuse temporary objects (reduce GC pressure)
// Objects may be garbage collected at any time (no guarantee)
// Good for: byte slices, buffers, expensive-to-allocate but short-lived objects

var bufPool = sync.Pool{
    New: func() interface{} {
        return make([]byte, 0, 4096)
    },
}

func processRequest(data []byte) []byte {
    buf := bufPool.Get().([]byte)  // get from pool or allocate
    buf = buf[:0]                   // reset length, keep capacity
    defer bufPool.Put(buf)         // return to pool

    buf = append(buf, data...)
    buf = transform(buf)

    // Make a copy before returning (buf goes back to pool!)
    result := make([]byte, len(buf))
    copy(result, buf)
    return result
}

// sync.Pool in fmt package: reduces allocations for formatting
// sync.Pool in encoding/json: reuses encoder state

// WARNING: don't store pointers to resources that need cleanup
// Pool objects may be evicted by GC at any time
// Don't assume pool objects are zero-valued (reset before use)
```

Q76

What is sync.Cond?

Difficulty: Hard

```go
// sync.Cond: condition variable (wait for a condition to become true)
// Used when goroutines need to wait for state changes

type Queue struct {
    mu    sync.Mutex
    cond  *sync.Cond
    items []int
}

func NewQueue() *Queue {
    q := &Queue{}
    q.cond = sync.NewCond(&q.mu)
    return q
}

func (q *Queue) Put(item int) {
    q.mu.Lock()
    q.items = append(q.items, item)
    q.mu.Unlock()
    q.cond.Signal()    // wake one waiter
    // q.cond.Broadcast() // wake all waiters
}

func (q *Queue) Get() int {
    q.mu.Lock()
    defer q.mu.Unlock()
    for len(q.items) == 0 {
        q.cond.Wait()  // releases lock, waits, re-acquires lock
    }
    item := q.items[0]
    q.items = q.items[1:]
    return item
}

// In practice: channels are preferred over sync.Cond
// sync.Cond: when you need broadcast semantics or complex state checks
```

Q77

What is the actor model pattern in Go?

Difficulty: Hard

```go
// Actor: goroutine that owns its state, communicates via channels only
// No shared mutable state → no locks needed

type AccountActor struct {
    balance  float64
    requests chan Request
    done     chan struct{}
}

type Request struct {
    Type     string
    Amount   float64
    Response chan<- Response
}

type Response struct {
    Balance float64
    Err     error
}

func (a *AccountActor) Start(ctx context.Context) {
    go func() {
        for {
            select {
            case req := <-a.requests:
                a.handleRequest(req)
            case <-ctx.Done():
                return
            }
        }
    }()
}

func (a *AccountActor) handleRequest(req Request) {
    switch req.Type {
    case "deposit":
        a.balance += req.Amount
        req.Response <- Response{Balance: a.balance}
    case "withdraw":
        if req.Amount > a.balance {
            req.Response <- Response{Err: errors.New("insufficient funds")}
            return
        }
        a.balance -= req.Amount
        req.Response <- Response{Balance: a.balance}
    case "balance":
        req.Response <- Response{Balance: a.balance}
    }
}
```

Q78
What is the mutex vs channel decision?

Difficulty: Medium

```go
Use Mutex when:
  - Protecting shared state (counter, map, cache)
  - Ownership doesn't transfer
  - Simple critical sections
  - Performance critical (mutex < channel overhead)

Use Channel when:
  - Passing ownership of data between goroutines
  - Coordinating goroutine lifecycle
  - Signaling events
  - Pipeline patterns
  - Fan-out / fan-in

Rob Pike's guideline:
  "Use communication to share memory,
   don't share memory to communicate"

In practice:
  Counter with 1000 goroutines: atomic.AddInt64 (fastest)
  Cache with reads >> writes: sync.RWMutex
  Job queue: buffered channel
  Result collection: channel or mutex-protected slice
  Lifecycle: context + done channel
```

Q79

What is the publish-subscribe pattern in Go?

Difficulty: Hard

```go
type PubSub struct {
    mu          sync.RWMutex
    subscribers map[string][]chan interface{}
}

func NewPubSub() *PubSub {
    return &PubSub{
        subscribers: make(map[string][]chan interface{}),
    }
}

func (ps *PubSub) Subscribe(topic string) <-chan interface{} {
    ch := make(chan interface{}, 10)
    ps.mu.Lock()
    ps.subscribers[topic] = append(ps.subscribers[topic], ch)
    ps.mu.Unlock()
    return ch
}

func (ps *PubSub) Publish(topic string, msg interface{}) {
    ps.mu.RLock()
    subs := ps.subscribers[topic]
    ps.mu.RUnlock()

    for _, ch := range subs {
        select {
        case ch <- msg:
        default:  // skip slow subscribers (drop message)
        }
    }
}

func (ps *PubSub) Unsubscribe(topic string, ch <-chan interface{}) {
    ps.mu.Lock()
    defer ps.mu.Unlock()
    subs := ps.subscribers[topic]
    for i, s := range subs {
        if s == ch {
            ps.subscribers[topic] = append(subs[:i], subs[i+1:]...)
            close(s)
            return
        }
    }
}
```

Q80

What is a rate limiter pattern in Go?

Difficulty: Hard

```go
import "golang.org/x/time/rate"

// Token bucket rate limiter
limiter := rate.NewLimiter(rate.Limit(100), 10)
// 100 tokens/sec, burst of 10

// Wait (blocks until token available)
if err := limiter.Wait(ctx); err != nil {
    return err  // context cancelled
}
doWork()

// Try (non-blocking)
if !limiter.Allow() {
    return errors.New("rate limit exceeded")
}
doWork()

// Reserve (get token in advance)
r := limiter.Reserve()
time.Sleep(r.Delay())
doWork()

// Per-user rate limiting with map
type RateLimiterMap struct {
    mu       sync.Mutex
    limiters map[string]*rate.Limiter
}

func (m *RateLimiterMap) Get(key string) *rate.Limiter {
    m.mu.Lock()
    defer m.mu.Unlock()
    if l, ok := m.limiters[key]; ok { return l }
    l := rate.NewLimiter(rate.Limit(10), 3)
    m.limiters[key] = l
    return l
}
```

---

5. Runtime Internals & Memory

Q81

What is the GMP scheduler?

Difficulty: Hard

```go
G - Goroutine: lightweight thread of execution
M - Machine: OS thread
P - Processor: scheduling context (GOMAXPROCS = number of Ps)

Structure:
  P has: local run queue (LRQ) of goroutines
  M runs: one G at a time, needs P to run goroutines
  Global run queue (GRQ): overflow from LRQs

Scheduling:
  1. G created → placed in P's LRQ
  2. M asks P for G to run
  3. G runs until: yield, syscall, preemption, channel block
  4. G blocks: M parks goroutine, picks another from LRQ
  5. Work stealing: idle P steals half of busy P's LRQ

Syscall:
  G makes blocking syscall → M detaches from P
  P acquires another M (or creates new one) to keep Ps busy
  When syscall completes: M tries to reacquire P

Preemption:
  Go 1.14+: asynchronous preemption (signal-based)
  Goroutine running long computation is preempted at safe points
  Prior: only preempted at function calls (starvation possible)

GOMAXPROCS:
  Default: runtime.NumCPU()
  Set: runtime.GOMAXPROCS(n) or GOMAXPROCS=n env
```

Q82

What is Go's garbage collector?

Difficulty: Hard

```go
Go GC: concurrent, tri-color mark-and-sweep

Algorithm (simplified):
  1. STW (Stop The World): enable write barriers, root scanning
  2. Mark (concurrent): traverse from roots, mark reachable objects
     - Tri-color: white (unreachable), grey (found, children not checked), black (done)
  3. STW: finish marking
  4. Sweep (concurrent): reclaim unmarked (white) memory

Write barriers:
  During marking: mutator (app) writes → write barrier records changes
  Ensures concurrent marking sees all pointers

Generational hypothesis: short-lived objects die young
  Go GC is NOT generational (different from JVM)
  All objects same treatment (no young/old gen)
  Go 1.17+: considering generational addition

GC tuning:
  GOGC=100 (default): GC when heap grows 100% since last GC
  GOGC=off: disable GC (for benchmarks)
  GOGC=200: more memory, fewer GC pauses
  runtime.GC(): force GC
  debug.SetGCPercent(): runtime adjustment

GC pauses:
  Go 1.5+: sub-millisecond STW
  Go 1.14+: typically < 500 microseconds
  Improve with: reduce allocations, sync.Pool, pre-allocate
```

Q83

What is escape analysis?

Difficulty: Hard

```go
// Escape analysis: compiler determines if variable lives on stack or heap

// Stack allocation: function-local, cheap, no GC pressure
func stackOnly() {
    x := 42          // stays on stack (no references escape)
    y := [100]int{}  // array on stack if doesn't escape
    _ = x; _ = y
}

// Heap allocation: escapes stack
func heapEscapes() *int {
    x := 42
    return &x  // x escapes to heap (pointer returned)
}

func heapEscapes2() {
    s := make([]int, 10000000)  // too large for stack → heap
    _ = s
}

func heapEscapes3() {
    var i interface{} = 42  // boxing: value may escape
}

// Check escape analysis
go build -gcflags="-m -m" ./...
// Output:
// ./main.go:5:6: x escapes to heap
// ./main.go:9:12: make([]int, 10000000) escapes to heap

// Reduce heap allocations for performance:
// Pass by value for small structs
// Reuse buffers (sync.Pool)
// Avoid interface boxing in hot paths
// Pre-allocate slices/maps with known size
```

Q84

What are stack vs heap trade-offs?

Difficulty: Hard

```go
// Stack pros:
//   - No GC involvement (just move stack pointer)
//   - Cache friendly (sequential memory)
//   - Thread-local (no synchronization)
// Stack cons:
//   - Size limited (default 8MB per thread in Java; 2KB-1GB in Go)
//   - Cannot return pointers to stack-allocated variables
//     (Go allows this via escape analysis + heap allocation)

// Heap pros:
//   - Unlimited size (until OOM)
//   - Can share between goroutines
//   - Can outlive the function that created it
// Heap cons:
//   - GC pressure (allocation, scanning, collection)
//   - Fragmentation
//   - Slower allocation (needs GC metadata)

// Goroutine stacks in Go:
//   - Start at 2KB (Go 1.4+), grow as needed
//   - Growth via stack copying (not segmented stacks since Go 1.4)
//   - Maximum: 1GB (configurable via runtime/debug.SetMaxStack)
//   - Stack grows when frame pointer would overflow current stack
//     → allocate larger stack, copy all frames, update pointers

// Practical optimization:
// Alloc 1M small structs: prefer sync.Pool or slab allocator
// Large temporary buffer: sync.Pool of []byte
// String manipulation: strings.Builder (single allocation)
```

Q85

What is GOMAXPROCS and when should you tune it?

Difficulty: Medium

```go
import "runtime"

// Get current GOMAXPROCS
n := runtime.GOMAXPROCS(0)  // 0 = query without changing

// Set GOMAXPROCS
runtime.GOMAXPROCS(4)  // use 4 OS threads

// Default: runtime.NumCPU() (number of logical CPUs)

// When to tune down:
//   - Container with limited CPU quota
//   - Avoid CPU-hungry app starving other processes
//   - CPU-bound workload in shared environment

// When to tune up: rarely needed
//   Go already uses all CPUs by default

// IMPORTANT for containers:
//   GOMAXPROCS defaults to host CPUs, not container limit!
//   Container says "2 CPUs" but GOMAXPROCS might be 64 (host)
//   Fix: github.com/uber-go/automaxprocs
import _ "go.uber.org/automaxprocs"  // auto-sets based on CPU quota

// Or manually:
if quota := getCPUQuota(); quota > 0 {
    runtime.GOMAXPROCS(int(math.Ceil(quota)))
}
```

Q86

How do you use pprof for profiling?

Difficulty: Hard

```go
// CPU profile
import _ "net/http/pprof"  // auto-registers /debug/pprof/

// Add to HTTP server:
go http.ListenAndServe(":6060", nil)

// Collect profile:
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30

// In pprof interactive mode:
// top10     - top 10 functions by CPU
// web       - open flame graph in browser
// list main - show annotated source
// traces    - show goroutine traces

// Manual profiling in code:
f, _ := os.Create("cpu.prof")
pprof.StartCPUProfile(f)
defer pprof.StopCPUProfile()
// ... run code ...

// Memory profile
f, _ := os.Create("mem.prof")
pprof.WriteHeapProfile(f)

// Goroutine profile
f, _ := os.Create("goroutine.prof")
pprof.Lookup("goroutine").WriteTo(f, 0)

// Benchmark with profile
go test -cpuprofile cpu.prof -memprofile mem.prof -bench=. ./...
go tool pprof cpu.prof
go tool pprof mem.prof

// Trace (detailed timeline)
f, _ := os.Create("trace.out")
trace.Start(f)
defer trace.Stop()
go tool trace trace.out
```

Q87

What are memory allocation patterns to avoid?

Difficulty: Hard

```go
// 1. Concatenating strings in loop (O(n²) allocations)
var result string
for _, s := range strs {
    result += s  // allocates new string each iteration
}
// Fix:
var b strings.Builder
for _, s := range strs { b.WriteString(s) }
result := b.String()

// 2. Appending without pre-allocation
var s []int
for i := 0; i < 10000; i++ {
    s = append(s, i)  // many reallocations (1→2→4→8...→16384)
}
// Fix:
s := make([]int, 0, 10000)  // pre-allocate

// 3. Map without size hint
m := make(map[string]int)  // many rehashes
// Fix:
m := make(map[string]int, 1000)  // pre-allocate ~1000 entries

// 4. Interface boxing in hot path
func sum(vals []interface{}) int {  // []interface{}: heap alloc per element
// Fix: use specific type or generics
func sum[T ~int](vals []T) T { ... }

// 5. Pointer to small value
p := new(int)       // heap allocation!
*p = 42
// Fix: use value directly
x := 42

// 6. Closures capturing large variables
data := make([]byte, 1<<20)  // 1MB
f := func() { process(data) }  // data stays alive as long as f
// Fix: pass data as parameter if closure outlives data usage
```

Q88

What is finalizer in Go?

Difficulty: Hard

```go
// Finalizer: function called before GC collects an object
// Use for: releasing C resources, closing OS handles

type Resource struct {
    handle unsafe.Pointer
}

func NewResource() *Resource {
    r := &Resource{handle: C.open_resource()}
    runtime.SetFinalizer(r, func(r *Resource) {
        C.close_resource(r.handle)
    })
    return r
}

// Problems with finalizers:
// - Non-deterministic timing (GC decides when)
// - Can delay GC (finalizer must run before collection)
// - Cannot guarantee order
// - Resurrect objects (finalizer can make object reachable again)

// Prefer: explicit Close() method + defer/context
// Use finalizer only as SAFETY NET, not primary cleanup

// Remove finalizer
runtime.SetFinalizer(r, nil)

// Better pattern: explicit lifecycle
type Resource struct {
    handle unsafe.Pointer
    once   sync.Once
}

func (r *Resource) Close() {
    r.once.Do(func() { C.close_resource(r.handle) })
}
```

Q89

What is the runtime package's key functionality?

Difficulty: Medium

```go
import "runtime"

// Goroutine info
runtime.NumGoroutine()          // current goroutine count
runtime.Goexit()                // terminate current goroutine
runtime.Gosched()               // yield scheduler (cooperative)
runtime.LockOSThread()          // pin goroutine to current OS thread
runtime.UnlockOSThread()

// CPU info
runtime.NumCPU()                // number of logical CPUs
runtime.GOMAXPROCS(n)           // set/get max parallel threads

// Memory
runtime.GC()                    // force GC
runtime.ReadMemStats(&stats)    // heap alloc, GC stats
runtime.KeepAlive(x)            // prevent premature GC of x

// Caller info (for logging/tracing)
_, file, line, ok := runtime.Caller(0)   // current function
_, file, line, ok := runtime.Caller(1)   // caller's caller
pcs := make([]uintptr, 10)
n := runtime.Callers(0, pcs)
frames := runtime.CallersFrames(pcs[:n])

// Stack trace
buf := make([]byte, 4096)
n := runtime.Stack(buf, false)  // false=current goroutine only
n := runtime.Stack(buf, true)   // true=all goroutines
```

Q90

What is memory leaks in Go and how to detect them?

Difficulty: Hard

```go
// Common memory leaks:

// 1. Goroutine leak (most common)
// Goroutine stuck forever → holds references → GC can't collect

// 2. Slice holding large array
big := make([]byte, 1<<30)  // 1GB
small := big[:10]           // small holds reference to big!
// GC cannot collect 1GB even though we only use 10 bytes
// Fix:
small := make([]byte, 10)
copy(small, big[:10])
big = nil  // release reference

// 3. Map that grows but never shrinks
cache := make(map[string]string)
for ever { cache[key] = value }  // grows unbounded
// Fix: LRU cache with max size, or TTL-based cleanup

// 4. Timer/ticker not stopped
ticker := time.NewTicker(time.Second)
// ... use ticker ...
// ticker.Stop() // MUST be called or goroutine leaks

// 5. Context values holding large data
ctx = context.WithValue(ctx, key, largeObject)  // lives until ctx cancelled

// Detection:
// 1. pprof heap profile (before/after)
// 2. Grafana/Prometheus: process_resident_memory_bytes growing
// 3. runtime.ReadMemStats: HeapAlloc growing without GC reclaiming
// 4. goleak: detect goroutine leaks in tests
```

Q91-Q100: More Concurrency & Runtime

Q91
What is runtime.LockOSThread()?

```go
// LockOSThread: pin current goroutine to its OS thread
// Use when: calling C code that uses thread-local storage
//           using OS-level APIs that need same thread
//           OpenGL, GTK (UI must be on same thread)

func worker() {
    runtime.LockOSThread()
    defer runtime.UnlockOSThread()
    // ... C code, UI calls, etc.
}
```

Q92
What is the go scheduler work stealing?

```go
Work stealing:
  Each P has local run queue (LRQ)
  When P's LRQ is empty:
    1. Try to steal half of another P's LRQ
    2. Check global run queue (GRQ)
    3. Check network poller (goroutines waiting on I/O)

  Goal: keep all Ps busy, minimize idle CPUs
  Result: near-optimal CPU utilization automatically

  FIFO local queue, LIFO steal (newer tasks first)
  Avoids context switching by keeping goroutines on same P
```

Q93

What is goroutine stack growth (contiguous stacks)?

```go
Go 1.4+: contiguous stacks (vs segmented stacks in earlier versions)

Growth:
  Default size: 2KB-8KB
  When stack would overflow:
    1. Allocate new, larger stack (2x current)
    2. Copy entire stack to new location
    3. Update all pointers (stack scanning)
    4. Continue execution

Shrink:
  Stacks shrink during GC if under-utilized
  Prevents long-lived goroutines hoarding memory

Problem solved:
  "Hot split" in segmented stacks: function on boundary
  called in tight loop → constant stack growth/shrink
  Contiguous: no split penalty
```

Q94

What is debug.SetMemoryLimit (Go 1.19+)?

```go
import "runtime/debug"

// Soft memory limit: hint to GC to be more aggressive near limit
debug.SetMemoryLimit(512 * 1024 * 1024)  // 512MB soft limit

// GC will run more frequently to stay under limit
// Prevents OOM in containers with memory limits

// GOMEMLIMIT environment variable
// GOMEMLIMIT=512MiB go run main.go

// Combine with GOGC for tuning:
// GOGC=off GOMEMLIMIT=512MiB: don't GC unless near limit
// Reduces GC overhead when memory is ample
```

Q95

What is runtime/trace package?

```go
import "runtime/trace"

// Execution trace: goroutine events, heap changes, GC events
f, _ := os.Create("trace.out")
trace.Start(f)
defer trace.Stop()

// View with:
go tool trace trace.out
// Shows: goroutine timelines, blocking events, GC activity

// In tests:
go test -trace trace.out ./...

// HTTP endpoint
import _ "net/http/pprof"
// GET http://localhost:6060/debug/pprof/trace?seconds=5
// wget -O trace.out http://localhost:6060/debug/pprof/trace?seconds=5
// go tool trace trace.out
```

Q96

What is the difference between runtime.Gosched and sync primitives?

```go
// runtime.Gosched(): cooperative yield — give other goroutines a chance to run
// Does NOT block, just yields current goroutine's time slice

// Use case: CPU-bound tight loop that doesn't call other functions
// (no natural preemption points before Go 1.14)
func tightLoop() {
    for i := 0; i < 1e9; i++ {
        if i % 1000 == 0 {
            runtime.Gosched()  // yield periodically
        }
        doWork()
    }
}

// Go 1.14+: asynchronous preemption → Gosched rarely needed
// But still useful for explicit fairness in tight loops

// vs sync primitives:
// sync.Mutex.Lock(): blocks goroutine (puts to sleep)
// runtime.Gosched(): yields but stays runnable
// channel op: blocks until ready
```

Q97

How does Go handle OS signals?

```go
import (
    "os"
    "os/signal"
    "syscall"
)

// Graceful shutdown on SIGTERM/SIGINT
func main() {
    sigs := make(chan os.Signal, 1)
    signal.Notify(sigs, syscall.SIGTERM, syscall.SIGINT)

    done := make(chan struct{})
    go func() {
        sig := <-sigs
        log.Printf("received signal: %v", sig)

        // Graceful shutdown
        server.Shutdown(context.Background())
        close(done)
    }()

    server.ListenAndServe()
    <-done
    log.Println("server stopped")
}

// signal.Reset: unregister signals
signal.Reset(syscall.SIGTERM)

// signal.Stop: stop delivering to channel
signal.Stop(sigs)
```

Q98

What is the heap profiler and how to interpret it?

```bash
# Collect heap profile
go tool pprof http://localhost:6060/debug/pprof/heap

# Interactive commands:
# top                 - top memory allocators
# top -cum            - including children (cumulative)
# list functionName   - annotated source
# web                 - flame graph
# svg > graph.svg     - save graph

# Allocation types in pprof:
# alloc_objects: total objects allocated (even if freed)
# alloc_space:   total bytes allocated
# inuse_objects: currently live objects
# inuse_space:   currently live bytes

# Focus on inuse_space for memory leaks
# Focus on alloc_space for GC pressure

# Example flags:
go tool pprof -alloc_space http://localhost:6060/debug/pprof/heap
go tool pprof -inuse_space http://localhost:6060/debug/pprof/heap
```

Q99

What is the benchmark framework in Go?

```go
func BenchmarkSum(b *testing.B) {
    data := make([]int, 1000)
    for i := range data { data[i] = i }

    b.ResetTimer()  // exclude setup from benchmark
    b.ReportAllocs() // report allocation count

    for i := 0; i < b.N; i++ {  // b.N auto-adjusted for accuracy
        sum(data)
    }
}

// Run benchmarks:
go test -bench=. ./...
go test -bench=BenchmarkSum -benchmem ./...
// Output:
// BenchmarkSum-8    1000000    1203 ns/op    0 B/op    0 allocs/op

// Sub-benchmarks:
func BenchmarkSort(b *testing.B) {
    for _, size := range []int{100, 1000, 10000} {
        b.Run(fmt.Sprintf("size=%d", size), func(b *testing.B) {
            data := makeData(size)
            b.ResetTimer()
            for i := 0; i < b.N; i++ {
                sort.Ints(data)
            }
        })
    }
}

// Compare benchmarks:
go test -bench=. -count=5 | tee old.txt
# ... change code ...
go test -bench=. -count=5 | tee new.txt
benchstat old.txt new.txt
```

Q100

What are cgo and its performance implications?

```go
// cgo: call C code from Go
/*
#include <stdlib.h>
#include <string.h>

char* greet(const char* name) {
    char* result = malloc(100);
    snprintf(result, 100, "Hello, %s!", name);
    return result;
}
*/
import "C"

func GoGreet(name string) string {
    cName := C.CString(name)
    defer C.free(unsafe.Pointer(cName))

    result := C.greet(cName)
    defer C.free(unsafe.Pointer(result))
    return C.GoString(result)
}

// cgo COSTS:
// - Each cgo call: ~200ns overhead (vs ~1ns for Go function call)
// - Transition between Go and C stacks
// - GC must stop for cgo calls in certain modes
// - Disables some Go tooling (race detector, pprof have limitations)

// When to use cgo:
// - Wrap existing C library (OpenSSL, SQLite, etc.)
// - Performance-critical C code that's hard to reimplement

// Avoid cgo when: pure Go alternative exists
// CGO_ENABLED=0: disable cgo entirely (static binary)
```

---

6. Error Handling & Testing

Q101

What is Go's error handling philosophy?

Difficulty: Medium

```go
// Errors are values — not exceptions
// Handle at the call site, not somewhere above

// Basic pattern
result, err := doSomething()
if err != nil {
    return fmt.Errorf("doSomething failed: %w", err)  // wrap with context
}

// Sentinel errors
var (
    ErrNotFound     = errors.New("not found")
    ErrUnauthorized = errors.New("unauthorized")
    ErrBadInput     = errors.New("bad input")
)

// Check specific error
if errors.Is(err, ErrNotFound) {
    // handle not found
}

// Custom error type
type AppError struct {
    Code    int
    Message string
    Cause   error
}
func (e *AppError) Error() string { return fmt.Sprintf("[%d] %s", e.Code, e.Message) }
func (e *AppError) Unwrap() error { return e.Cause }

// Check error type
var appErr *AppError
if errors.As(err, &appErr) {
    fmt.Println("app error code:", appErr.Code)
}
```

Q102

What is errors.Is vs errors.As?

Difficulty: Medium

```go
// errors.Is: checks if ANY error in the chain equals target (by value)
// errors.As: checks if ANY error in the chain is of type target (by type)

var ErrNotFound = errors.New("not found")

// Chain: wrappedErr → ErrNotFound
wrappedErr := fmt.Errorf("get user: %w", ErrNotFound)

// errors.Is traverses chain
errors.Is(wrappedErr, ErrNotFound)  // true
wrappedErr == ErrNotFound           // false (different pointer)

// errors.As: find specific type in chain
type DatabaseError struct{ Code int; Message string }
func (e *DatabaseError) Error() string { return e.Message }

dbErr := &DatabaseError{Code: 1001, Message: "connection failed"}
wrapped := fmt.Errorf("query users: %w", dbErr)

var dbErrTarget *DatabaseError
if errors.As(wrapped, &dbErrTarget) {
    fmt.Println("DB error code:", dbErrTarget.Code)  // 1001
}

// Implement custom Is for equality check
type NotFoundError struct{ Resource string; ID int64 }
func (e *NotFoundError) Is(target error) bool {
    if t, ok := target.(*NotFoundError); ok {
        return t.Resource == e.Resource  // match by resource type only
    }
    return false
}
```

Q103

What are the testing package basics?

Difficulty: Easy

```go
// Test file: must end in _test.go
// Test function: must start with Test, take *testing.T
package mypackage

import "testing"

func TestAdd(t *testing.T) {
    result := Add(2, 3)
    if result != 5 {
        t.Errorf("Add(2,3) = %d, want 5", result)
    }
}

// Fail methods:
t.Error("msg")         // log + mark fail, continue
t.Errorf("fmt", args)  // formatted, continue
t.Fatal("msg")         // log + mark fail, STOP test (calls t.FailNow)
t.Fatalf("fmt", args)  // formatted, stop
t.Log("msg")           // log (shown only on failure)
t.Logf("fmt", args)    // formatted log

// Run tests:
go test ./...             // all packages
go test -v ./...          // verbose
go test -run TestAdd ./... // specific test
go test -count=1 ./...    // disable caching
go test -race ./...       // race detector
```

Q104

What are table-driven tests?

Difficulty: Medium

```go
func TestAdd(t *testing.T) {
    cases := []struct {
        name     string
        a, b     int
        expected int
    }{
        {"positive", 2, 3, 5},
        {"negative", -1, -2, -3},
        {"zero", 0, 0, 0},
        {"mixed", -1, 1, 0},
    }

    for _, tc := range cases {
        tc := tc  // capture range variable (pre-Go 1.22)
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel()  // run subtests in parallel
            got := Add(tc.a, tc.b)
            if got != tc.expected {
                t.Errorf("Add(%d, %d) = %d, want %d", tc.a, tc.b, got, tc.expected)
            }
        })
    }
}

// Run specific subtest:
// go test -run TestAdd/positive ./...
// go test -run TestAdd/. ./...  (all subtests)
```

Q105

What is testify?

Difficulty: Easy

```go
import (
    "testing"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "github.com/stretchr/testify/mock"
)

func TestSomething(t *testing.T) {
    // assert: continues on failure
    assert.Equal(t, 5, Add(2, 3))
    assert.NotNil(t, result)
    assert.NoError(t, err)
    assert.Error(t, err)
    assert.Contains(t, "hello world", "world")
    assert.Len(t, slice, 3)
    assert.True(t, condition, "should be true because...")

    // require: stops test on failure (like t.Fatal)
    require.NoError(t, err)  // test stops here if err != nil
    require.NotNil(t, result)

    // assert.EqualValues: works across compatible numeric types
    assert.EqualValues(t, 5, int64(5))
}

// Mock with testify/mock
type MockUserRepo struct{ mock.Mock }

func (m *MockUserRepo) GetUser(ctx context.Context, id int64) (*User, error) {
    args := m.Called(ctx, id)
    return args.Get(0).(*User), args.Error(1)
}

mockRepo := new(MockUserRepo)
mockRepo.On("GetUser", mock.Anything, int64(1)).Return(&User{ID: 1}, nil)
```

Q106

What is httptest package?

Difficulty: Medium

```go
import (
    "net/http"
    "net/http/httptest"
    "testing"
)

func TestHandler(t *testing.T) {
    handler := http.HandlerFunc(MyHandler)

    // Create test request
    req := httptest.NewRequest(http.MethodGet, "/users/42", nil)
    req.Header.Set("Authorization", "Bearer token")

    // Create response recorder
    rr := httptest.NewRecorder()

    // Call handler
    handler.ServeHTTP(rr, req)

    // Assert response
    assert.Equal(t, http.StatusOK, rr.Code)
    assert.Equal(t, "application/json", rr.Header().Get("Content-Type"))

    var user User
    json.Unmarshal(rr.Body.Bytes(), &user)
    assert.Equal(t, int64(42), user.ID)
}

// Test entire server
server := httptest.NewServer(router)
defer server.Close()

resp, err := http.Get(server.URL + "/health")
assert.Equal(t, 200, resp.StatusCode)

// TLS server
server := httptest.NewTLSServer(router)
client := server.Client()  // pre-configured to trust test cert
```

Q107

What are test fixtures and test helpers?

Difficulty: Medium

```go
// Test helper: prefixed with helper, calls t.Helper()
func assertEqual(t *testing.T, got, want interface{}) {
    t.Helper()  // marks this as helper (failure shows caller's line)
    if got != want {
        t.Errorf("got %v, want %v", got, want)
    }
}

// Test fixture: setup common test data
func setupTestDB(t *testing.T) *sql.DB {
    t.Helper()
    db, err := sql.Open("postgres", testDSN)
    require.NoError(t, err)

    // Run migrations
    runMigrations(db)

    // Cleanup on test end
    t.Cleanup(func() {
        db.Exec("DELETE FROM orders")
        db.Close()
    })
    return db
}

func TestCreateOrder(t *testing.T) {
    db := setupTestDB(t)  // setup + auto-cleanup
    repo := NewOrderRepository(db)
    // test code
}

// TestMain: global setup/teardown
func TestMain(m *testing.M) {
    // Setup
    setupTestDatabase()

    code := m.Run()  // run all tests

    // Teardown
    teardownTestDatabase()

    os.Exit(code)
}
```

Q108

What is fuzz testing in Go (1.18+)?

Difficulty: Medium

```go
// Fuzz test: automatically generates random inputs to find bugs
func FuzzReverse(f *testing.F) {
    // Seed corpus: known interesting inputs
    f.Add("hello")
    f.Add("")
    f.Add("a")
    f.Add("■■■")

    f.Fuzz(func(t *testing.T, orig string) {
        reversed := Reverse(orig)
        doubleReversed := Reverse(reversed)

        // Invariant: double reverse = original
        if orig != doubleReversed {
            t.Errorf("Reverse(%q)=%q, DoubleReverse=%q", orig, reversed, doubleReversed)
        }

        // Invariant: reverse of valid UTF-8 should also be valid UTF-8
        if utf8.ValidString(orig) && !utf8.ValidString(reversed) {
            t.Errorf("Reverse(%q) is not valid UTF-8", orig)
        }
    })
}

// Run fuzz test (finds new crashes):
go test -fuzz=FuzzReverse -fuzztime=60s ./...

// Run with corpus only (regression test):
go test -run=FuzzReverse ./...

// Found crashes stored in testdata/fuzz/FuzzReverse/
```

Q109

What is error wrapping and unwrapping?

Difficulty: Medium

```go
// Wrap error with context
err := fmt.Errorf("get user %d: %w", id, originalErr)  // %w wraps
// or
err := fmt.Errorf("get user: %w", originalErr)

// Unwrap single level
unwrapped := errors.Unwrap(err)  // one level

// Walk entire chain with errors.Is and errors.As

// Custom error with Unwrap
type QueryError struct {
    Query string
    Err   error
}
func (e *QueryError) Error() string { return fmt.Sprintf("query %q: %v", e.Query, e.Err) }
func (e *QueryError) Unwrap() error { return e.Err }  // enables Is/As traversal

// Error chain:
// QueryError → DatabaseError → sql.ErrNoRows
qErr := &QueryError{Query: "SELECT...", Err: &DatabaseError{Err: sql.ErrNoRows}}
errors.Is(qErr, sql.ErrNoRows)  // true (walks chain)
```

Q110

What are benchmarks and microbenchmarks?

Difficulty: Medium

```go
// Avoid benchmark pitfalls:

// 1. Compiler optimization elimination (use result)
func BenchmarkAdd(b *testing.B) {
    var result int
    for i := 0; i < b.N; i++ {
        result = Add(i, i)  // use result to prevent elimination
    }
    _ = result
}

// 2. Reset timer after setup
func BenchmarkSort(b *testing.B) {
    data := generateData(10000)
    b.ResetTimer()  // don't count setup
    for i := 0; i < b.N; i++ {
        sort.Ints(data)
    }
}

// 3. Parallel benchmark
func BenchmarkConcurrent(b *testing.B) {
    b.RunParallel(func(pb *testing.PB) {
        for pb.Next() {
            atomicCounter.Add(1)
        }
    })
}

// Profile benchmark
go test -bench=BenchmarkSort -cpuprofile cpu.prof -memprofile mem.prof
go tool pprof cpu.prof
```

---

7. Standard Library & HTTP

Q111

What is the net/http package?

Difficulty: Medium

```go
// HTTP server
mux := http.NewServeMux()
mux.HandleFunc("/users", getUsers)
mux.HandleFunc("/users/", getUserByID)

server := &http.Server{
    Addr:         ":8080",
    Handler:      mux,
    ReadTimeout:  5 * time.Second,
    WriteTimeout: 10 * time.Second,
    IdleTimeout:  120 * time.Second,
}
log.Fatal(server.ListenAndServe())

// Handler
func getUsers(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }

    users, err := userService.List(r.Context())
    if err != nil {
        http.Error(w, "internal error", http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(users)
}

// Graceful shutdown
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt)
defer stop()

go server.ListenAndServe()
<-ctx.Done()
server.Shutdown(context.Background())
```

Q112

What are HTTP middleware patterns?

Difficulty: Hard

```go
// Middleware: wraps http.Handler with additional behavior
type Middleware func(http.Handler) http.Handler

// Auth middleware
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        if token == "" {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }
        userID, err := validateToken(token)
        if err != nil {
            http.Error(w, "invalid token", http.StatusUnauthorized)
            return
        }
        // Add user to context
        ctx := context.WithValue(r.Context(), userIDKey, userID)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// Logging middleware
func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        wrapped := &responseWriter{ResponseWriter: w, status: 200}
        next.ServeHTTP(wrapped, r)
        log.Printf("%s %s %d %v", r.Method, r.URL.Path, wrapped.status, time.Since(start))
    })
}

// Chain middleware
func Chain(h http.Handler, middlewares ...Middleware) http.Handler {
    for i := len(middlewares) - 1; i >= 0; i-- {
        h = middlewares[i](h)
    }
    return h
}

handler := Chain(mux, LoggingMiddleware, AuthMiddleware, RecoverMiddleware)
```

Q113

What is encoding/json?

Difficulty: Medium

```go
// Marshal (Go → JSON)
type User struct {
    ID        int64     `json:"id"`
    Name      string    `json:"name"`
    Email     string    `json:"email"`
    Password  string    `json:"-"`              // excluded
    CreatedAt time.Time `json:"created_at"`
    Score     *float64  `json:"score,omitempty"` // omit if nil
    Tags      []string  `json:"tags,omitempty"`  // omit if empty
}

data, err := json.Marshal(user)
// Pretty print:
data, err := json.MarshalIndent(user, "", "  ")

// Unmarshal (JSON → Go)
var u User
err := json.Unmarshal(data, &u)

// Streaming (efficient for large data)
enc := json.NewEncoder(w)
enc.Encode(user)  // writes to w

dec := json.NewDecoder(r.Body)
dec.Decode(&user)  // reads from r.Body

// Custom marshal/unmarshal
func (u *User) MarshalJSON() ([]byte, error) {
    type Alias User
    return json.Marshal(&struct {
        *Alias
        CreatedAt string `json:"created_at"`
    }{
        Alias:     (*Alias)(u),
        CreatedAt: u.CreatedAt.Format(time.RFC3339),
    })
}
```

Q114

What is the context package for HTTP?

Difficulty: Medium

```go
// Each HTTP request has a context (available via r.Context())
// Context is cancelled when: client disconnects, handler returns, timeout

func handler(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()

    // Pass context to all downstream calls
    user, err := userRepo.GetUser(ctx, userID)  // DB query respects cancellation
    if err != nil {
        if errors.Is(err, context.Canceled) {
            // Client disconnected, no need to respond
            return
        }
        http.Error(w, "error", 500)
        return
    }

    // Add request-scoped values
    requestID := uuid.New().String()
    ctx = context.WithValue(ctx, requestIDKey, requestID)

    // Set deadline for this request's processing
    ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    result, err := processWithContext(ctx, data)
    // ...
}

// Extract value from context
func getRequestID(ctx context.Context) string {
    if id, ok := ctx.Value(requestIDKey).(string); ok {
        return id
    }
    return ""
}
```

Q115

What is the io package and streaming?

Difficulty: Medium

```go
// io.Copy: efficient copy between Reader and Writer
// Uses 32KB buffer internally
n, err := io.Copy(dst, src)

// io.LimitReader: read at most N bytes
limited := io.LimitReader(r.Body, 1<<20)  // 1MB max
data, err := io.ReadAll(limited)

// io.TeeReader: read and copy simultaneously
var buf bytes.Buffer
tee := io.TeeReader(src, &buf)  // reads from src AND copies to buf
io.Copy(dst, tee)
// buf now contains everything that was read

// io.Pipe: synchronous in-memory pipe
pr, pw := io.Pipe()
go func() {
    defer pw.Close()
    json.NewEncoder(pw).Encode(data)  // write to pipe
}()
http.Post(url, "application/json", pr)  // read from pipe

// bufio: buffered I/O
reader := bufio.NewReader(r)
line, err := reader.ReadString('\n')
scanner := bufio.NewScanner(r)
for scanner.Scan() { process(scanner.Text()) }

writer := bufio.NewWriter(w)
writer.WriteString("hello\n")
writer.Flush()  // don't forget!
```

Q116

What is the time package?

Difficulty: Easy

```go
// Current time
now := time.Now()            // local time
utc := time.Now().UTC()      // UTC

// Duration
d := 5 * time.Second
d := 100 * time.Millisecond
d := time.Hour + 30*time.Minute

// Time operations
future := now.Add(24 * time.Hour)
past   := now.Add(-7 * 24 * time.Hour)
diff   := future.Sub(past)    // time.Duration

// Comparison
before := t1.Before(t2)
after  := t1.After(t2)
equal  := t1.Equal(t2)  // use Equal, not == (handles timezone)

// Formatting
now.Format(time.RFC3339)           // "2024-01-15T10:30:00Z"
now.Format("2006-01-02 15:04:05")  // custom (Go's reference time!)
now.Unix()                          // Unix timestamp (seconds)
now.UnixMilli()                     // milliseconds

// Parsing
t, err := time.Parse(time.RFC3339, "2024-01-15T10:30:00Z")
t, err := time.Parse("2006-01-02", "2024-01-15")

// Sleep and timer
time.Sleep(time.Second)
timer := time.NewTimer(5 * time.Second)
ticker := time.NewTicker(time.Minute)
defer timer.Stop()
defer ticker.Stop()
```

Q117

What is the strings and strconv package?

Difficulty: Easy

```go
import "strings"

strings.Contains("hello", "ell")      // true
strings.HasPrefix("hello", "hel")     // true
strings.HasSuffix("hello", "llo")     // true
strings.Count("hello", "l")           // 2
strings.Index("hello", "ll")          // 2, -1 if not found
strings.Replace("oink oink", "k", "ky", 1) // "oinky oink" (1 replacement)
strings.ReplaceAll("oink oink", "k", "ky") // "oinky oinky"
strings.ToLower("HELLO")              // "hello"
strings.ToUpper("hello")              // "HELLO"
strings.TrimSpace("  hello  ")        // "hello"
strings.Trim("--hello--", "-")        // "hello"
strings.TrimPrefix("hello", "hel")    // "lo"
strings.TrimSuffix("hello", "llo")    // "he"
strings.Split("a,b,c", ",")           // ["a","b","c"]
strings.Join([]string{"a","b"}, ", ") // "a, b"
strings.Fields("  foo bar  baz  ")    // ["foo","bar","baz"]
strings.Repeat("ab", 3)               // "ababab"

import "strconv"
strconv.Itoa(42)                      // "42"
n, err := strconv.Atoi("42")         // 42
f, err := strconv.ParseFloat("3.14", 64)
i, err := strconv.ParseInt("-42", 10, 64)
u, err := strconv.ParseUint("42", 10, 64)
b, err := strconv.ParseBool("true")
strconv.FormatFloat(3.14, 'f', 2, 64) // "3.14"
```

Q118
What is gRPC in Go?

Difficulty: Hard

```protobuf
// service.proto
syntax = "proto3";
service UserService {
    rpc GetUser(GetUserRequest) returns (User);
    rpc ListUsers(ListUsersRequest) returns (stream User);
    rpc CreateUser(stream CreateUserRequest) returns (CreateUserResponse);
}
message GetUserRequest { int64 id = 1; }
message User { int64 id = 1; string name = 2; string email = 3; }
```

```go
// Server implementation
type userServer struct {
    pb.UnimplementedUserServiceServer
    repo UserRepository
}

func (s *userServer) GetUser(ctx context.Context, req *pb.GetUserRequest) (*pb.User, error) {
    user, err := s.repo.GetUser(ctx, req.Id)
    if err != nil {
        return nil, status.Errorf(codes.NotFound, "user %d not found", req.Id)
    }
    return &pb.User{Id: user.ID, Name: user.Name, Email: user.Email}, nil
}

// gRPC server
srv := grpc.NewServer(
    grpc.UnaryInterceptor(authInterceptor),
    grpc.StreamInterceptor(streamAuthInterceptor),
)
pb.RegisterUserServiceServer(srv, &userServer{})
lis, _ := net.Listen("tcp", ":50051")
srv.Serve(lis)

// Client
conn, _ := grpc.DialContext(ctx, "localhost:50051",
    grpc.WithTransportCredentials(insecure.NewCredentials()),
    grpc.WithUnaryInterceptor(retryInterceptor),
)
client := pb.NewUserServiceClient(conn)
user, err := client.GetUser(ctx, &pb.GetUserRequest{Id: 42})
```

Q119

What is the os and filepath package?

Difficulty: Easy

```go
import "os"
import "path/filepath"

// Environment
os.Getenv("HOME")
os.Setenv("KEY", "value")
os.LookupEnv("KEY")  // returns (value, bool)

// Files
f, err := os.Open("file.txt")         // read-only
f, err := os.Create("file.txt")       // write-only, truncate
f, err := os.OpenFile("file.txt", os.O_RDWR|os.O_CREATE, 0644)
data, err := os.ReadFile("file.txt")  // read entire file
err = os.WriteFile("file.txt", data, 0644)

// Directory
err = os.Mkdir("dir", 0755)
err = os.MkdirAll("a/b/c", 0755)   // create all parents
err = os.Remove("file.txt")
err = os.RemoveAll("dir")           // recursive
entries, err := os.ReadDir(".")
for _, e := range entries {
    fmt.Println(e.Name(), e.IsDir())
}

// filepath
filepath.Join("a", "b", "c")        // "a/b/c" (OS-specific sep)
filepath.Base("/a/b/file.txt")       // "file.txt"
filepath.Dir("/a/b/file.txt")        // "/a/b"
filepath.Ext("file.txt")             // ".txt"
filepath.Abs("./rel/path")           // absolute path

// Walk directory
filepath.Walk(".", func(path string, info os.FileInfo, err error) error {
    fmt.Println(path)
    return nil
})
```

Q120

What is the log and slog package?

Difficulty: Easy

```go
// Standard log package (simple)
log.Println("server started")
log.Printf("error: %v", err)
log.Fatal("fatal error")  // log + os.Exit(1)

// slog package (Go 1.21+) - structured logging
import "log/slog"

// Default logger
slog.Info("user created", "user_id", 42, "email", "alice@example.com")
slog.Error("database error", "err", err, "query", sql)
slog.Debug("processing", "items", len(items))

// JSON handler for production
logger := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelInfo,
}))
slog.SetDefault(logger)

// With context (add common fields)
logger = logger.With("service", "user-api", "version", "1.0")
logger.Info("request", "method", r.Method, "path", r.URL.Path, "status", 200)

// Custom handler for Datadog/OpenTelemetry integration
// Third-party: zerolog, zap, logrus
```

Q121-Q125: More Standard Library

Q121

What is sync/errgroup vs WaitGroup?

```go
// WaitGroup: no error propagation
// errgroup: collects and returns first error

// errgroup (golang.org/x/sync/errgroup)
g, ctx := errgroup.WithContext(context.Background())

g.Go(func() error {
    return fetchURL(ctx, "http://example.com/1")
})
g.Go(func() error {
    return fetchURL(ctx, "http://example.com/2")
})

// Wait for all AND get first error
if err := g.Wait(); err != nil {
    log.Fatal(err)
}
// ctx is also cancelled when first error occurs
```

Q122

What is the math/rand vs crypto/rand?

```go
// math/rand: pseudo-random, deterministic, FAST
// DO NOT use for security/crypto!
r := rand.New(rand.NewSource(time.Now().UnixNano()))
n := r.Intn(100)         // [0, 100)
f := r.Float64()         // [0.0, 1.0)

// crypto/rand: cryptographically secure, slower
// USE for: tokens, passwords, keys, IDs
import "crypto/rand"
token := make([]byte, 32)
rand.Read(token)
tokenStr := hex.EncodeToString(token)
// or: base64.URLEncoding.EncodeToString(token)

// Random UUID using crypto/rand
id, _ := uuid.NewRandom()  // uses crypto/rand internally
```

Q123

What is the bytes package?

```go
// bytes: like strings but for []byte

import "bytes"

bytes.Contains(data, []byte("hello"))
bytes.HasPrefix(data, []byte("GET "))
bytes.Equal(a, b)                        // compare
bytes.Count(data, []byte("go"))
bytes.Replace(data, []byte("old"), []byte("new"), -1)
bytes.Split(data, []byte(","))
bytes.TrimSpace(data)
bytes.ToLower(data)

// bytes.Buffer: efficient concatenation
var buf bytes.Buffer
buf.WriteString("hello")
buf.WriteByte(' ')
buf.Write([]byte("world"))
result := buf.Bytes()  // or buf.String()
buf.Reset()            // reset for reuse

// Compare to strings.Builder:
// bytes.Buffer: read and write
// strings.Builder: write only, returns string
```

Q124

What is the encoding package family?

```go
import "encoding/base64"
import "encoding/hex"
import "encoding/csv"

// Base64
encoded := base64.StdEncoding.EncodeToString(data)
decoded, _ := base64.StdEncoding.DecodeString(encoded)
// URL-safe (for tokens in URLs)
encoded := base64.URLEncoding.EncodeToString(data)

// Hex
hexStr := hex.EncodeToString(data)
decoded, _ := hex.DecodeString(hexStr)

// CSV
reader := csv.NewReader(r)
records, _ := reader.ReadAll()
for _, row := range records { fmt.Println(row) }

writer := csv.NewWriter(w)
writer.Write([]string{"id", "name", "email"})
writer.Flush()
```

Q125
What is the net package?

```go
// TCP server
listener, _ := net.Listen("tcp", ":8080")
for {
    conn, _ := listener.Accept()
    go handleConn(conn)
}

func handleConn(conn net.Conn) {
    defer conn.Close()
    buf := make([]byte, 1024)
    n, _ := conn.Read(buf)
    conn.Write(buf[:n])  // echo
}

// TCP client
conn, _ := net.Dial("tcp", "localhost:8080")
defer conn.Close()
conn.Write([]byte("hello"))

// UDP
conn, _ := net.ListenPacket("udp", ":9090")
buf := make([]byte, 1024)
n, addr, _ := conn.ReadFrom(buf)
conn.WriteTo(buf[:n], addr)

// DNS lookup
ips, _ := net.LookupIP("google.com")
addrs, _ := net.LookupHost("google.com")
```

8. Design Patterns in Go

Q126

What is the functional options pattern?

Difficulty: Medium

```go
type Server struct {
    host    string
    port    int
    timeout time.Duration
    maxConn int
    tls     bool
}

type Option func(*Server)

func WithHost(h string) Option     { return func(s *Server) { s.host = h } }
func WithPort(p int) Option        { return func(s *Server) { s.port = p } }
func WithTimeout(t time.Duration) Option { return func(s *Server) { s.timeout = t } }
func WithMaxConn(n int) Option     { return func(s *Server) { s.maxConn = n } }
func WithTLS() Option              { return func(s *Server) { s.tls = true } }

func NewServer(opts ...Option) *Server {
    s := &Server{
        host:    "localhost",
        port:    8080,
        timeout: 30 * time.Second,
        maxConn: 100,
    }
    for _, opt := range opts {
        opt(s)
    }
    return s
}

srv := NewServer(
    WithPort(9090),
    WithTimeout(time.Minute),
    WithTLS(),
)
```

Q127

What is the repository pattern?

Difficulty: Medium

```go
// Repository: abstracts data storage
type UserRepository interface {
    GetByID(ctx context.Context, id int64) (*User, error)
    GetByEmail(ctx context.Context, email string) (*User, error)
    Create(ctx context.Context, user *User) error
    Update(ctx context.Context, user *User) error
    Delete(ctx context.Context, id int64) error
    List(ctx context.Context, filter UserFilter) ([]*User, error)
}

// PostgreSQL implementation
type postgresUserRepo struct {
    db *pgxpool.Pool
}

func (r *postgresUserRepo) GetByID(ctx context.Context, id int64) (*User, error) {
    var u User
    err := r.db.QueryRow(ctx,
        "SELECT id,name,email,created_at FROM users WHERE id=$1 AND deleted_at IS NULL",
        id,
    ).Scan(&u.ID, &u.Name, &u.Email, &u.CreatedAt)
    if errors.Is(err, pgx.ErrNoRows) {
        return nil, ErrNotFound
    }
    return &u, err
}

// Service depends on interface, not concrete impl
type UserService struct {
    repo   UserRepository  // injected
    cache  Cache
    events EventPublisher
}

func NewUserService(repo UserRepository, cache Cache, events EventPublisher) *UserService {
    return &UserService{repo: repo, cache: cache, events: events}
}
```

Q128

What is dependency injection in Go?

Difficulty: Medium

```go
// DI: inject dependencies via constructor, not instantiate inside
// Benefits: testable, configurable, loosely coupled

// Bad: hidden dependency
type UserService struct{}
func (s *UserService) GetUser(id int64) (*User, error) {
    db := getGlobalDB()  // hidden dependency!
    return db.Query(...)
}

// Good: explicit dependency injection
type UserService struct {
    repo   UserRepository
    logger *slog.Logger
    cache  Cache
}

func NewUserService(repo UserRepository, logger *slog.Logger, cache Cache) *UserService {
    return &UserService{repo: repo, logger: logger, cache: cache}
}

// Wire (google/wire): compile-time DI code generation
//go:generate wire

// wire.go
func InitializeApp(ctx context.Context) (*App, func(), error) {
    wire.Build(
        NewPostgresPool,
        NewUserRepository,
        NewUserService,
        NewHTTPServer,
        NewApp,
    )
    return nil, nil, nil
}
```

Q129

What is the middleware chain pattern?

Difficulty: Medium

```go
// Handler type
type HandlerFunc func(ctx context.Context, req Request) (Response, error)

// Middleware type
type Middleware func(HandlerFunc) HandlerFunc

// Logging middleware
func WithLogging(logger *slog.Logger) Middleware {
    return func(next HandlerFunc) HandlerFunc {
        return func(ctx context.Context, req Request) (Response, error) {
            start := time.Now()
            resp, err := next(ctx, req)
            logger.Info("request",
                "method", req.Method,
                "duration", time.Since(start),
                "error", err,
            )
            return resp, err
        }
    }
}

// Retry middleware
func WithRetry(maxAttempts int) Middleware {
    return func(next HandlerFunc) HandlerFunc {
        return func(ctx context.Context, req Request) (Response, error) {
            var err error
            for i := 0; i < maxAttempts; i++ {
                var resp Response
                resp, err = next(ctx, req)
                if err == nil { return resp, nil }
                time.Sleep(time.Duration(i+1) * 100 * time.Millisecond)
            }
            return Response{}, err
        }
    }
}

// Chain
func Chain(h HandlerFunc, middlewares ...Middleware) HandlerFunc {
    for i := len(middlewares) - 1; i >= 0; i-- {
        h = middlewares[i](h)
    }
    return h
}
```

Q130

What is the circuit breaker pattern in Go?

Difficulty: Hard

```go
import "github.com/sony/gobreaker"

type CircuitBreakerService struct {
    cb   *gobreaker.CircuitBreaker
    next Service
}

func NewCircuitBreakerService(next Service) *CircuitBreakerService {
    cb := gobreaker.NewCircuitBreaker(gobreaker.Settings{
        Name:        "external-service",
        MaxRequests: 3,            // half-open: allow 3 requests
        Interval:    time.Minute,  // reset counts every minute
        Timeout:     30 * time.Second, // open → half-open after 30s
        ReadyToTrip: func(c gobreaker.Counts) bool {
            return c.ConsecutiveFailures > 5  // open after 5 consecutive failures
        },
        OnStateChange: func(name string, from, to gobreaker.State) {
            log.Printf("circuit breaker %s: %s → %s", name, from, to)
        },
    })
    return &CircuitBreakerService{cb: cb, next: next}
}

func (s *CircuitBreakerService) Call(ctx context.Context, req Request) (Response, error) {
    result, err := s.cb.Execute(func() (interface{}, error) {
        return s.next.Call(ctx, req)
    })
    if err != nil {
        if errors.Is(err, gobreaker.ErrOpenState) {
            return Response{}, ErrServiceUnavailable
        }
        return Response{}, err
    }
    return result.(Response), nil
}
```

Q131

What is the builder pattern?

Difficulty: Medium

```go
type QueryBuilder struct {
    table      string
    conditions []string
    orderBy    string
    limit      int
    args       []interface{}
}

func NewQuery(table string) *QueryBuilder {
    return &QueryBuilder{table: table}
}

func (q *QueryBuilder) Where(condition string, args ...interface{}) *QueryBuilder {
    q.conditions = append(q.conditions, condition)
    q.args = append(q.args, args...)
    return q
}

func (q *QueryBuilder) OrderBy(field string) *QueryBuilder {
    q.orderBy = field
    return q
}

func (q *QueryBuilder) Limit(n int) *QueryBuilder {
    q.limit = n
    return q
}

func (q *QueryBuilder) Build() (string, []interface{}) {
    query := "SELECT * FROM " + q.table
    if len(q.conditions) > 0 {
        query += " WHERE " + strings.Join(q.conditions, " AND ")
    }
    if q.orderBy != "" { query += " ORDER BY " + q.orderBy }
    if q.limit > 0 { query += fmt.Sprintf(" LIMIT %d", q.limit) }
    return query, q.args
}

// Usage (method chaining)
sql, args := NewQuery("users").
    Where("age > $1", 18).
    Where("active = $2", true).
    OrderBy("name").
    Limit(10).
    Build()
```

Q132

What is the observer/event pattern?

Difficulty: Hard

```go
type EventType string
const (
    UserCreated EventType = "user.created"
    OrderPlaced EventType = "order.placed"
)

type Event struct {
    Type    EventType
    Payload interface{}
}

type Handler func(ctx context.Context, e Event) error

type EventBus struct {
    mu       sync.RWMutex
    handlers map[EventType][]Handler
}

func (b *EventBus) Subscribe(eventType EventType, h Handler) {
    b.mu.Lock()
    defer b.mu.Unlock()
    b.handlers[eventType] = append(b.handlers[eventType], h)
}

func (b *EventBus) Publish(ctx context.Context, e Event) error {
    b.mu.RLock()
    handlers := b.handlers[e.Type]
    b.mu.RUnlock()

    for _, h := range handlers {
        if err := h(ctx, e); err != nil {
            return err
        }
    }
    return nil
}

// Usage
bus.Subscribe(UserCreated, sendWelcomeEmail)
bus.Subscribe(UserCreated, createUserProfile)
bus.Subscribe(OrderPlaced, chargePayment)

bus.Publish(ctx, Event{Type: UserCreated, Payload: user})
```

Q133

What is the template method pattern?

Difficulty: Medium

```go
// Template method: define skeleton algorithm, subclasses fill in steps
// In Go: use interfaces or function injection

type DataProcessor interface {
    Read() ([]byte, error)
    Process([]byte) ([]byte, error)
    Write([]byte) error
}

// Template function
func RunPipeline(dp DataProcessor) error {
    data, err := dp.Read()
    if err != nil { return fmt.Errorf("read: %w", err) }

    processed, err := dp.Process(data)
    if err != nil { return fmt.Errorf("process: %w", err) }

    if err := dp.Write(processed); err != nil {
        return fmt.Errorf("write: %w", err)
    }
    return nil
}

// Implementation
type CSVProcessor struct{
    inputPath, outputPath string
}
func (p *CSVProcessor) Read() ([]byte, error) { return os.ReadFile(p.inputPath) }
func (p *CSVProcessor) Process(d []byte) ([]byte, error) { return transform(d), nil }
func (p *CSVProcessor) Write(d []byte) error { return os.WriteFile(p.outputPath, d, 0644) }

RunPipeline(&CSVProcessor{inputPath: "in.csv", outputPath: "out.csv"})
```

Q134

What is the strategy pattern?

Difficulty: Medium

```go
// Strategy: interchangeable algorithms behind an interface
type SortStrategy interface {
    Sort(data []int) []int
}

type QuickSort struct{}
func (q QuickSort) Sort(data []int) []int { /* quicksort */ return data }

type MergeSort struct{}
func (m MergeSort) Sort(data []int) []int { /* mergesort */ return data }

type Sorter struct {
    strategy SortStrategy
}

func (s *Sorter) SetStrategy(strategy SortStrategy) {
    s.strategy = strategy
}

func (s *Sorter) Sort(data []int) []int {
    return s.strategy.Sort(data)
}

// Dynamic strategy selection
sorter := &Sorter{}
if len(data) < 100 {
    sorter.SetStrategy(QuickSort{})
} else {
    sorter.SetStrategy(MergeSort{})
}
result := sorter.Sort(data)

// In Go: often simpler as function type
type SortFunc func([]int) []int

func sortWith(data []int, strategy SortFunc) []int {
    return strategy(data)
}
```

Q135

What is the command pattern?

Difficulty: Medium

```go
// Command: encapsulate operation as an object (undo/redo, queuing)
type Command interface {
    Execute() error
    Undo() error
}

type CreateUserCommand struct {
    user *User
    repo UserRepository
}

func (c *CreateUserCommand) Execute() error {
    return c.repo.Create(context.Background(), c.user)
}

func (c *CreateUserCommand) Undo() error {
    return c.repo.Delete(context.Background(), c.user.ID)
}

// Command history for undo/redo
type CommandHistory struct {
    history []Command
}

func (h *CommandHistory) Execute(cmd Command) error {
    if err := cmd.Execute(); err != nil { return err }
    h.history = append(h.history, cmd)
    return nil
}

func (h *CommandHistory) Undo() error {
    if len(h.history) == 0 { return errors.New("nothing to undo") }
    last := h.history[len(h.history)-1]
    h.history = h.history[:len(h.history)-1]
    return last.Undo()
}
```

---

9. Performance, Profiling & Production

Q136

What are common Go performance optimizations?

Difficulty: Hard

```go
// 1. Pre-allocate slices and maps
items := make([]Item, 0, expectedLen)  // avoid reallocations
cache := make(map[string]int, 1000)    // avoid rehashing

// 2. Avoid unnecessary string conversions
// BAD: allocates new string
key := string(byteSlice) + suffix
// GOOD: work with bytes directly, convert once
key := append(byteSlice, []byte(suffix)...)

// 3. Reuse buffers with sync.Pool
var bufPool = sync.Pool{New: func() interface{} { return new(bytes.Buffer) }}

func handler(w http.ResponseWriter, r *http.Request) {
    buf := bufPool.Get().(*bytes.Buffer)
    buf.Reset()
    defer bufPool.Put(buf)
    // use buf
}

// 4. Avoid interface boxing in hot path
// BAD: boxing allocates
for _, v := range largeSlice {
    sum += v.(int)  // unbox each iteration
}
// GOOD: typed slice
for _, v := range typedSlice { sum += v }

// 5. Use streaming instead of loading all into memory
// BAD:
data, _ := io.ReadAll(resp.Body)
json.Unmarshal(data, &result)
// GOOD:
json.NewDecoder(resp.Body).Decode(&result)

// 6. Inline hot functions (compiler does this, help with small funcs)
//go:noinline  // prevent inlining (for profiling clarity)
```

Q137

What is the go tool and useful commands?

Difficulty: Easy

```bash
# Build
go build ./...              # build all packages
go build -o myapp main.go   # build binary
go install ./...            # build and install to $GOPATH/bin

# Test
go test ./...               # test all packages
go test -v -run TestName    # verbose, specific test
go test -race -count=1 ./... # race detector, no cache
go test -bench=. -benchmem  # benchmarks with memory stats
go test -cover ./...        # coverage summary
go test -coverprofile=c.out # coverage profile

# Profile
go test -cpuprofile cpu.prof -bench=.
go tool pprof cpu.prof

# Vet (static analysis)
go vet ./...

# Format
gofmt -w .                  # format in place
goimports -w .              # format + fix imports

# Dependencies
go mod init module-name     # initialize module
go mod tidy                 # add missing, remove unused
go get package@version      # add dependency
go mod vendor               # copy deps to vendor/

# Documentation
go doc fmt.Println          # view docs
godoc -http=:6060           # browse all docs

# Linker flags (reduce binary size)
go build -ldflags="-s -w" -o myapp  # strip debug info
```

Q138

What is go mod and module system?

Difficulty: Easy

```go
// go.mod: module definition
module github.com/myorg/myapp

go 1.21

require (
    github.com/jackc/pgx/v5 v5.5.0
    github.com/redis/go-redis/v9 v9.3.0
    go.uber.org/zap v1.26.0
)

require (
    // indirect dependencies
    github.com/jackc/pgconn v1.14.0 // indirect
)

// replace: use local version
replace github.com/myorg/shared => ../shared

// go.sum: checksums for security
// Never edit manually

// Semantic versioning
// v1.0.0: initial stable
// v1.2.3: major.minor.patch
// v2.0.0: breaking change → module path changes to /v2

// Upgrade
go get github.com/some/pkg@latest  // latest version
go get github.com/some/pkg@v1.2.3  // specific version
go mod tidy                        // clean up
```

Q139

What is goroutine best practices for production?

Difficulty: Hard

```go
// Always ensure goroutines can exit
// Always pass context for cancellation
// Always handle panics in goroutines

func safeGo(ctx context.Context, fn func(ctx context.Context)) {
    go func() {
        defer func() {
            if r := recover(); r != nil {
                log.Printf("recovered panic: %v\n%s", r, debug.Stack())
            }
        }()
        fn(ctx)
    }()
}

// Limit goroutine count
// Use worker pools, semaphores, or errgroup.SetLimit

// Don't fire-and-forget without tracking
type GoroutineTracker struct {
    wg sync.WaitGroup
}

func (t *GoroutineTracker) Go(fn func()) {
    t.wg.Add(1)
    go func() {
        defer t.wg.Done()
        fn()
    }()
}

func (t *GoroutineTracker) Wait() { t.wg.Wait() }

// Set goroutine names for debugging
// pprof.SetGoroutineLabels
pprof.Do(ctx, pprof.Labels("name", "order-processor"), func(ctx context.Context) {
    processOrders(ctx)
})
```

Q140

What are go linters and static analysis tools?

Difficulty: Easy

```yaml
# .golangci.yml
linters:
  enable:
    - errcheck      # check error returns
    - gosimple      # simplification suggestions
    - govet         # suspicious constructs
    - ineffassign   # unused assignments
    - staticcheck   # advanced static analysis
    - unused        # unused code
    - gocyclo       # cyclomatic complexity
    - misspell      # spelling
    - gosec         # security issues
    - revive        # general linting
    - noctx         # HTTP requests without context

linters-settings:
  gocyclo:
    min-complexity: 15
  errcheck:
    check-type-assertions: true
```

```bash
# Run linters
golangci-lint run ./...
golangci-lint run --fix ./...  # auto-fix where possible

# Specific tools
staticcheck ./...
go vet ./...
shadow ./...  # variable shadowing
bodyclose ./... # HTTP response body not closed
```

Q141

What are production-ready HTTP server best practices?

Difficulty: Hard

```go
func NewServer(cfg Config) *http.Server {
    mux := http.NewServeMux()
    registerRoutes(mux)

    handler := chain(mux,
        requestIDMiddleware,
        loggingMiddleware,
        recoveryMiddleware,
        rateLimitMiddleware,
        corsMiddleware,
    )

    return &http.Server{
        Addr:    cfg.Addr,
        Handler: handler,

        // Timeouts (MUST set to prevent resource exhaustion)
        ReadTimeout:       5 * time.Second,    // header + body read
        ReadHeaderTimeout: 2 * time.Second,    // header only
        WriteTimeout:      10 * time.Second,   // response write
        IdleTimeout:       120 * time.Second,  // keep-alive timeout

        MaxHeaderBytes: 1 << 20,  // 1MB max header

        // TLS
        TLSConfig: &tls.Config{
            MinVersion:               tls.VersionTLS13,
            PreferServerCipherSuites: true,
        },
    }
}

// Graceful shutdown
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()

go server.ListenAndServe()
<-ctx.Done()

shutdownCtx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
server.Shutdown(shutdownCtx)
```

Q142

What are common Go interview questions about goroutines?

Difficulty: Hard

```go
// Q: What does this print?
wg := sync.WaitGroup{}
for i := 0; i < 3; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        fmt.Println(i)  // Go <1.22: prints 3,3,3 (loop variable capture)
    }()
}
wg.Wait()
// Fix: i := i  or  go func(i int) { ... }(i)
// Go 1.22+: prints 0,1,2 (each iteration has own variable)

// Q: What happens here?
ch := make(chan int)
go func() { ch <- 1 }()
go func() { ch <- 2 }()
fmt.Println(<-ch)
fmt.Println(<-ch)
// A: prints 1 and 2 (in some order - race between goroutines)

// Q: Does this deadlock?
ch := make(chan int)
ch <- 1          // A: YES, deadlock: unbuffered, no receiver
fmt.Println(<-ch)

// Q: What does this print?
var m sync.Mutex
m.Lock()
go func() {
    m.Lock()   // blocks
    fmt.Println("got lock")
    m.Unlock()
}()
m.Unlock()     // goroutine unblocks
time.Sleep(time.Millisecond)  // give goroutine time
// A: prints "got lock"
```

Q143

What is go workspace (go work)?

Difficulty: Medium

```bash
# go.work: multi-module workspace (Go 1.18+)
# Use when working on multiple related modules simultaneously

go work init ./myapp ./mylib

# go.work
go 1.21

use (
    ./myapp   # uses local version of myapp module
    ./mylib   # uses local version of mylib module
)

# Now changes to mylib are immediately visible to myapp
# without publishing a new version

# Build/test respects workspace
go build ./...  # builds all modules in workspace
go test ./...   # tests all modules

# Disable workspace (use module versions from go.mod)
GOWORK=off go build

# Vendor in workspaces
go work vendor  # create workspace-level vendor directory
```

Q144

What are Go's slice gotchas?

Difficulty: Hard

```go
// Gotcha 1: append may or may not modify original
s := []int{1, 2, 3}
t := s[:2]          // t shares backing array
t = append(t, 99)   // IF cap allows: modifies s[2]!
// s = [1, 2, 99]   // surprising!

// Fix: use full slice expression to set cap
t := s[:2:2]        // t has cap=2
t = append(t, 99)   // new backing array (doesn't affect s)

// Gotcha 2: range copies values
type User struct{ Name string }
users := []User{{Name: "Alice"}, {Name: "Bob"}}
for _, u := range users {
    u.Name = "Changed"  // modifies copy, not original
}
// users unchanged!
// Fix: use index
for i := range users { users[i].Name = "Changed" }

// Gotcha 3: nil vs empty slice in JSON
var s []int    → null in JSON
s := []int{}  → [] in JSON

// Gotcha 4: memory leak via slice
big := loadLargeData()  // 1GB
small := big[:10]       // holds reference to 1GB!
// Fix:
small := make([]byte, 10)
copy(small, big[:10])
big = nil
```

Q145

What is the embed package (Go 1.16+)?

Difficulty: Medium

```go
import "embed"

// Embed single file
//go:embed static/index.html
var indexHTML []byte

// Embed entire directory
//go:embed static
var staticFiles embed.FS

// Use embedded files
content, err := staticFiles.ReadFile("static/index.html")
entries, err := staticFiles.ReadDir("static")

// HTTP file server from embedded files
http.Handle("/static/", http.FileServer(http.FS(staticFiles)))

// Embed templates
//go:embed templates/*.html
var templates embed.FS

tmpl := template.Must(template.ParseFS(templates, "templates/*.html"))

// Migration files
//go:embed migrations/*.sql
var migrations embed.FS

// Benefits:
// - Single binary deployment (no external files needed)
// - Files baked into binary at compile time
// - Works with go:generate workflow
```

---

10. Advanced Go Patterns

Q146

What is the outbox pattern implementation in Go?

Difficulty: Hard

```go
// Transactional outbox: atomic DB write + eventual Kafka publish
type OutboxEntry struct {
    ID        string
    EventType string
    Payload   []byte
    CreatedAt time.Time
}

type OutboxRepository struct{ db *pgxpool.Pool }

// Write atomically within business transaction
func (r *OutboxRepository) Save(ctx context.Context, tx pgx.Tx, entry OutboxEntry) error {
    _, err := tx.Exec(ctx,
        "INSERT INTO outbox(id,event_type,payload,created_at) VALUES($1,$2,$3,$4)",
        entry.ID, entry.EventType, entry.Payload, entry.CreatedAt)
    return err
}

// Background poller: flush to Kafka
type OutboxPoller struct {
    repo     *OutboxRepository
    producer KafkaProducer
}

func (p *OutboxPoller) Poll(ctx context.Context) error {
    rows, err := p.repo.db.Query(ctx,
        `SELECT id,event_type,payload FROM outbox
         WHERE published_at IS NULL ORDER BY created_at LIMIT 100
         FOR UPDATE SKIP LOCKED`)
    if err != nil { return err }
    defer rows.Close()

    var ids []string
    for rows.Next() {
        var id, evtType string; var payload []byte
        rows.Scan(&id, &evtType, &payload)
        p.producer.Produce(evtType, payload)
        ids = append(ids, id)
    }
    if len(ids) > 0 {
        p.repo.db.Exec(ctx, "UPDATE outbox SET published_at=NOW() WHERE id=ANY($1)", ids)
    }
    return nil
}
```

Q147

What is graceful degradation with circuit breaker + fallback?

Difficulty: Hard

```go
type RecommendationService struct {
    primary  RecommendationClient
    fallback RecommendationClient  // simpler, always-available
    cb       *gobreaker.CircuitBreaker
}

func (s *RecommendationService) GetRecommendations(ctx context.Context, userID int64) ([]Item, error) {
    result, err := s.cb.Execute(func() (interface{}, error) {
        return s.primary.Get(ctx, userID)
    })

    if err != nil {
        // Circuit open or primary failed → use fallback (popular items)
        log.Printf("using fallback recommendations: %v", err)
        return s.fallback.Get(ctx, userID)
    }

    return result.([]Item), nil
}

// Multiple levels of fallback
func (s *Service) getData(ctx context.Context, id string) (Data, error) {
    // Level 1: Redis cache
    if data, err := s.cache.Get(ctx, id); err == nil { return data, nil }

    // Level 2: Primary DB
    if data, err := s.primaryDB.Get(ctx, id); err == nil {
        s.cache.Set(ctx, id, data, time.Hour)
        return data, nil
    }

    // Level 3: Read replica
    if data, err := s.replicaDB.Get(ctx, id); err == nil { return data, nil }

    // Level 4: Default/empty response
    return Data{ID: id, IsDefault: true}, nil
}
```

Q148-Q165: HTTP, gRPC, Database Patterns

Q148

What is HTTP/2 server push in Go?

```go
func handler(w http.ResponseWriter, r *http.Request) {
    pusher, ok := w.(http.Pusher)
    if ok {
        pusher.Push("/static/app.css", &http.PushOptions{
            Header: http.Header{"Content-Type": {"text/css"}},
        })
        pusher.Push("/static/app.js", nil)
    }
    // serve main page
}

// Enable HTTP/2 (requires TLS)
server := &http.Server{
    TLSConfig: &tls.Config{
        NextProtos: []string{"h2", "http/1.1"},
    },
}
server.ListenAndServeTLS("cert.pem", "key.pem")
```

Q149

What is SSE (Server-Sent Events) in Go?

```go
func sseHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")

    flusher, ok := w.(http.Flusher)
    if !ok { http.Error(w, "SSE not supported", 500); return }

    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-r.Context().Done():
            return
        case t := <-ticker.C:
            fmt.Fprintf(w, "data: %s\n\n", t.Format(time.RFC3339))
            flusher.Flush()
        }
    }
}
```

Q150

What is WebSocket in Go?

```go
import "github.com/gorilla/websocket"

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}

func wsHandler(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil { return }
    defer conn.Close()

    for {
        messageType, msg, err := conn.ReadMessage()
        if err != nil {
            if websocket.IsCloseError(err, websocket.CloseGoingAway) { return }
            log.Printf("read error: %v", err); return
        }
        conn.WriteMessage(messageType, msg)  // echo
    }
}
```

Q151

What is graceful HTTP server shutdown?

```go
server := &http.Server{Addr: ":8080", Handler: mux}

go func() {
    if err := server.ListenAndServe(); err != http.ErrServerClosed {
        log.Fatalf("server error: %v", err)
    }
}()

// Wait for interrupt
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := server.Shutdown(ctx); err != nil {
    log.Printf("forced shutdown: %v", err)
}
log.Println("server stopped")
```

Q152

What is the retry pattern with backoff?

```go
func withRetry(ctx context.Context, maxAttempts int, fn func() error) error {
    var err error
    for attempt := 0; attempt < maxAttempts; attempt++ {
        if err = fn(); err == nil { return nil }
        if !isRetryable(err) { return err }

        delay := time.Duration(math.Pow(2, float64(attempt))) * 100 * time.Millisecond
        jitter := time.Duration(rand.Intn(100)) * time.Millisecond

        select {
        case <-ctx.Done(): return ctx.Err()
        case <-time.After(delay + jitter):
        }
    }
    return fmt.Errorf("failed after %d attempts: %w", maxAttempts, err)
}
```

Q153

What is the health check pattern?

```go
type HealthChecker struct {
    db    *pgxpool.Pool
    redis *redis.Client
    kafka *kgo.Client
}

func (h *HealthChecker) Check(ctx context.Context) map[string]string {
    status := make(map[string]string)

    ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    if err := h.db.Ping(ctx); err != nil {
        status["postgres"] = "down: " + err.Error()
    } else {
        status["postgres"] = "up"
    }

    if err := h.redis.Ping(ctx).Err(); err != nil {
        status["redis"] = "down: " + err.Error()
    } else {
        status["redis"] = "up"
    }

    return status
}

func healthHandler(checker *HealthChecker) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        status := checker.Check(r.Context())
        w.Header().Set("Content-Type", "application/json")
        allOK := true
        for _, v := range status {
            if strings.HasPrefix(v, "down") { allOK = false; break }
        }
        if !allOK { w.WriteHeader(http.StatusServiceUnavailable) }
        json.NewEncoder(w).Encode(status)
    }
}
```

Q154-Q165: More Production Patterns

| Q | Topic |

|---|---|

| Q154 | Structured logging with slog and trace IDs |

| Q155 | OpenTelemetry tracing in Go services |

| Q156 | Prometheus metrics in Go (counter, gauge, histogram) |

| Q157 | gRPC interceptors (auth, logging, retry) |

| Q158 | go:generate workflow for mocks and stubs |

| Q159 | Go plugin system (build tags, interfaces) |

| Q160 | Kubernetes probes: liveness, readiness, startup |

| Q161 | Go build flags: CGO_ENABLED, GOOS, GOARCH |

| Q162 | Multi-stage Dockerfile for Go applications |

| Q163 | Go module proxy and private modules |

| Q164 | Dependency injection with google/wire |

| Q165 | Testing database code with testcontainers-go |

Q166

What is GORM vs sqlx vs pgx comparison?

Difficulty: Medium

```go
GORM:
  Pros: ORM, auto-migrations, associations, callbacks, hooks
  Cons: magic (N+1 hidden), performance overhead, complex queries awkward
  Use when: rapid prototyping, CRUD-heavy app

sqlx:
  Pros: thin wrapper over database/sql, struct scanning, named queries
  Cons: still reflection-based, no type safety, SQL as strings
  Use when: familiar with SQL, want struct scanning

pgx v5:
  Pros: PostgreSQL-specific, fastest, full feature support (COPY, arrays, JSONB)
  Cons: PostgreSQL only, more verbose
  Use when: performance matters, PostgreSQL only

sqlc:
  Pros: generates type-safe Go code from SQL, zero reflection
  Cons: code generation step, SQL-first workflow
  Use when: want type safety without ORM overhead

Recommendation: sqlc + pgx for production Go services
```

Q167-Q200: Additional Comprehensive Questions

Q167

What is the difference between time.Sleep and timer in a select?

```go
// time.Sleep: blocks goroutine, not cancellable
time.Sleep(5 * time.Second)  // cannot be interrupted by context

// time.After in select: cancellable
select {
case <-time.After(5 * time.Second):
    fmt.Println("timed out")
case <-ctx.Done():
    fmt.Println("cancelled")
}

// time.NewTimer: reusable, stoppable
timer := time.NewTimer(5 * time.Second)
defer timer.Stop()  // prevent leak
select {
case <-timer.C: doWork()
case <-ctx.Done(): return
}

// Reset timer correctly
if !timer.Stop() { <-timer.C }  // drain channel before reset
timer.Reset(5 * time.Second)
```

Q168

What is unsafe.Pointer and when is it used?

```go
import "unsafe"

// unsafe.Pointer: bypass type system (use with extreme caution)
// Convert between any pointer types

// Example: read struct as bytes (serialization hack)
type Data struct{ X, Y int64 }
d := Data{X: 1, Y: 2}
bytes := (*[16]byte)(unsafe.Pointer(&d))[:]

// atomic.Value for interface: use unsafe for nil check trick
// sync/atomic operations on non-standard types

// Allowed conversions:
// *T ↔ unsafe.Pointer
// unsafe.Pointer ↔ uintptr (for pointer arithmetic, briefly)
// unsafe.Pointer ↔ *U (for type reinterpretation)

// DO NOT: store uintptr (GC doesn't see it, pointer may move)
// DO: convert to unsafe.Pointer atomically
p := unsafe.Pointer(&x)  // OK
u := uintptr(p)          // DANGEROUS: GC may move x
```

Q169

What is the go:noescape and go:nosplit compiler directives?

```go
//go:noescape
// Prevents escape analysis from considering arguments to escape to heap
// Used in assembly-implemented functions
// Normal Go code doesn't need this

//go:nosplit
// Prevents stack splitting/growth for this function
// Use for functions called with very limited stack (signal handlers, etc.)

//go:noinline
// Prevents compiler from inlining this function
// Useful for profiling (inlined functions are invisible in profiles)

//go:linkname localname importpath.name
// Link to unexported symbol in another package (use with extreme caution)
```

Q170

What is the difference between os.Exit and panic?

```go
// os.Exit(code): immediate termination
//   - Defers DO NOT run
//   - Goroutines killed immediately
//   - No panic recovery possible
//   - Use in main() for fatal startup errors

os.Exit(1)  // exit with error code, no cleanup

// panic: unwinding termination
//   - Defers DO run (including recover())
//   - Can be recovered
//   - Use for programmer errors, unexpected states

panic("unreachable")
panic(fmt.Errorf("fatal: %w", err))

// log.Fatal = log.Print + os.Exit(1)
// log.Panic = log.Print + panic()

// Best practice:
// Use error returns for expected failures
// Use panic for programmer errors (nil pointer, out of bounds)
// Use os.Exit only in main() after cleanup
```

Q171

What is the net/http/httputil ReverseProxy?

```go
func NewReverseProxy(target string) *httputil.ReverseProxy {
    url, _ := url.Parse(target)
    proxy := httputil.NewSingleHostReverseProxy(url)

    proxy.ModifyResponse = func(resp *http.Response) error {
        resp.Header.Set("X-Proxied-By", "my-proxy")
        return nil
    }

    proxy.ErrorHandler = func(w http.ResponseWriter, r *http.Request, err error) {
        log.Printf("proxy error: %v", err)
        http.Error(w, "service unavailable", 503)
    }

    return proxy
}

// Use as middleware
mux.Handle("/api/", http.StripPrefix("/api", NewReverseProxy("http://backend:8081")))
```

Q172

What is sync.Map vs map with mutex performance comparison?

```go
Benchmarks (typical):
                        map+RWMutex    sync.Map
Read-heavy (99% read)   ~200ns         ~50ns    ← sync.Map wins
Write-heavy (50% write) ~100ns         ~300ns   ← map+RWMutex wins
Mixed (80% read)        ~180ns         ~80ns    ← sync.Map wins

sync.Map internals:
  - read-only fast path: atomic pointer swap (no lock)
  - write: acquires mutex, promotes to dirty map
  - Amortized cost: reads from promoted map are lock-free

Use sync.Map when:
  - Cache: read-heavy, infrequent writes
  - Key set mostly stable
  - Different goroutines access different keys (disjoint sets)

Use map+RWMutex when:
  - Frequent writes
  - Need Len()
  - Need to iterate frequently
```

Q173

What are Go generics constraints?

```go
// Builtin constraints
type Ordered interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64 |
    ~uint | ~uint8 | ~uint16 | ~uint32 | ~uint64 | ~uintptr |
    ~float32 | ~float64 |
    ~string
}

// golang.org/x/exp/constraints
import "golang.org/x/exp/constraints"

func Min[T constraints.Ordered](a, b T) T {
    if a < b { return a }
    return b
}

// Union types with ~: includes types with same underlying type
type Integer interface { ~int | ~int64 }
type MyInt int  // ~int includes MyInt

func Double[T Integer](v T) T { return v * 2 }
Double(42)         // works (int)
Double(MyInt(10))  // works (MyInt, underlying type is int)

// Type set interface (can't use as value type)
// Only usable as constraint
type Stringer interface { String() string }  // value type OK
type Number interface { ~int | ~float64 }    // constraint only
```

Q174

What is go:embed with templates?

```go
//go:embed templates
var templateFS embed.FS

func loadTemplates() *template.Template {
    return template.Must(template.New("").Funcs(template.FuncMap{
        "formatDate": func(t time.Time) string {
            return t.Format("2006-01-02")
        },
    }).ParseFS(templateFS, "templates/*.html"))
}

// Template file: templates/user.html
// {{template "base" .}}
// {{define "content"}}
//   <h1>{{.Name}}</h1>
// {{end}}

func renderTemplate(w http.ResponseWriter, name string, data interface{}) {
    tmpl := loadTemplates()
    if err := tmpl.ExecuteTemplate(w, name, data); err != nil {
        http.Error(w, "render error", 500)
    }
}
```

Q175
What is Go's memory model?

```go
Go Memory Model: rules for when reads see writes from another goroutine

Key rule: "happens-before" relationship
  A happens-before B: writes before A are visible to reads after B

Synchronization primitives establish happens-before:
  - Channel send happens-before corresponding receive
  - Channel close happens-before receive of zero value
  - sync.Mutex.Unlock happens-before next Lock
  - sync.WaitGroup.Done happens-before Wait return
  - sync.Once.Do return happens-before any Do invocation

Without synchronization: no guarantee!
  x := 0
  go func() { x = 1 }()
  fmt.Println(x)  // may see 0 or 1 (data race!)

Use race detector: go test -race
  Catches: concurrent reads/writes without sync primitives
```

Q176-Q200: Final 25 Questions

| Q | Topic |

|---|---|

| Q176 | Go interface best practices (small, focused) |

| Q177 | Nil pointer receiver: methods on nil pointers |

| Q178 | Comparing function values in Go |

| Q179 | Go's select fairness (random choice) |

| Q180 | Multiple return values and error wrapping patterns |

| Q181 | Struct tags (json, db, validate) |

| Q182 | Go enums with iota and String() method |

| Q183 | Type assertion vs type switch performance |

| Q184 | Avoiding allocations with unsafe string/bytes conversion |

| Q185 | go vet and common checks |

| Q186 | Testing with golden files |

| Q187 | Property-based testing with rapid |

| Q188 | Integration tests with testcontainers |

| Q189 | go build constraints for OS/arch |

| Q190 | CGO-free builds (CGO_ENABLED=0) |

| Q191 | Go program lifecycle: init, main, goroutines, defer, exit |

| Q192 | Implementing stringer with go generate |

| Q193 | Error sentinel vs error type trade-offs |

| Q194 | Context propagation in gRPC |

| Q195 | HTTP client best practices (timeouts, connection pooling) |

| Q196 | go/ast and code generation |

| Q197 | Reflection vs generics: when to use each |

| Q198 | Go program memory layout (stack frames, heap segments) |

| Q199 | go tool compile: understanding assembly output |

| Q200 | Production checklist: Go service readiness |

Master these 200 questions to ace any Go SDE2 interview. Key focus areas: concurrency (goroutines, channels, select, sync), runtime internals (GMP scheduler, GC, escape analysis), testing (table-driven, race detector, benchmarks), and production patterns (circuit breaker, retry, graceful shutdown). ■

Q171

What is Go's memory allocator (mcache, mcentral, mheap)?

```go
Go memory allocator: tcmalloc-inspired, lock-free for small objects

Size classes: 67 size classes (8B to 32KB)
  < 32KB: "small" object → size-classed allocation
  > 32KB: "large" object → directly from mheap

mcache (per-P, no lock):
  Each P has mcache: span cache per size class
  Allocation: bump pointer into span (O(1), no lock)
  Full span → get new span from mcentral

mcentral (per size class, mutex):
  Pool of spans for specific size class
  nonempty: spans with free objects
  empty: full spans

mheap (global, mutex):
  Manages virtual memory (64-bit: 512GB max)
  Spans of 8KB pages
  treap for free span management

Allocation path:
  P's mcache → mcentral (per size class) → mheap → OS

Tiny allocations (< 16B, no pointers):
  Multiple tiny objects packed into single 16B block
  Saves memory for strings, small structs

Go 1.17+: page allocator with radix tree for faster mheap operations
```

Q172

What is Go's defer optimization?

```go
// Go 1.14+: open-coded defers (zero overhead for simple cases)
// Compiler inlines defer at function exit points

// Fast defer (heap-allocated defer record avoided):
func fast() {
    defer unlock()  // open-coded: no allocation if < 8 defers
    doWork()
}

// defer in loop: not open-coded (dynamic, heap allocated)
for i := 0; i < n; i++ {
    defer cleanup(i)  // heap allocation per iteration
}

// defer with named return (modifies return value):
func f() (n int) {
    defer func() { n++ }()  // n is incremented after return
    return 1  // returns 2, not 1!
}

// Measure defer overhead:
// Open-coded defer: ~0ns (inlined)
// Heap-allocated defer: ~50ns
// Stack-allocated defer (Go 1.13): ~8ns

// Best practice: avoid defer in tight hot loops
// Use explicit cleanup calls instead for performance-critical code
```

Q173

What is goroutine local storage alternative in Go?

```go
// Go has no goroutine-local storage (by design — avoid hidden state)
// Alternatives:

// 1. context.Context (recommended)
type ctxKey struct{ name string }
ctx = context.WithValue(ctx, ctxKey{"requestID"}, "abc123")
requestID := ctx.Value(ctxKey{"requestID"}).(string)

// 2. Pass explicitly (cleanest)
func process(ctx context.Context, userID int64, db *DB) error { ... }

// 3. goroutine ID hack (not recommended)
// Can get goroutine ID via runtime stack parsing
// Brittle, unsupported, use only for debugging

// Why no goroutine-local storage?
// Goroutines are cheap — expected to create many
// goroutine-local state → hidden coupling
// context.Context is the Go way: explicit, cancellable
```

Q174
What is go:linkname directive?

```go
// go:linkname: link to unexported symbol in another package
// Use sparingly — bypasses package encapsulation!

package mypackage

import _ "unsafe"  // required for go:linkname

//go:linkname nanotime runtime.nanotime
func nanotime() int64  // declare signature, body in runtime

// Used by: standard library internal packages
// sync/atomic, time package access runtime internals

// Legitimate use: access runtime internals for performance
// sync.(*Mutex).state accessed via reflect in some test frameworks

// Better alternatives: always prefer exported APIs
// go:linkname couples you to internal implementation details
```

Q175
What is Go plugin system?

```go
// Go plugins: load .so files at runtime
// Limitations: same Go version, same build flags, Linux/macOS only

// Plugin file (plugin.go)
package main
import "fmt"
func Greet(name string) string { return fmt.Sprintf("Hello, %s!", name) }
// Build: go build -buildmode=plugin -o plugin.so plugin.go

// Host program
import "plugin"
p, err := plugin.Open("plugin.so")
f, err := p.Lookup("Greet")
greet := f.(func(string) string)
fmt.Println(greet("World"))

// Better alternatives in practice:
// 1. Interface + separate binary (microservices)
// 2. gRPC for cross-language plugins
// 3. WebAssembly (WASM) for sandboxed plugins
// 4. Hashicorp go-plugin: RPC-based plugins (used in Terraform, Vault)

// go-plugin advantages over native plugins:
// - Cross-version compatible
// - Crash isolation
// - Cross-language (gRPC)
// - Works on all platforms
```

Q176

What are common Go interview coding patterns?

```go
// Pattern 1: Two-pointer / Sliding window
func longestSubstring(s string, k int) int {
    maxLen := 0
    for _, char := range "abcdefghijklmnopqrstuvwxyz" {
        count := make(map[rune]int)
        left := 0
        for right, c := range s {
            count[c]++
            for count[char] > k { count[rune(s[left])]; left++ }
            if len(count) == k { maxLen = max(maxLen, right-left+1) }
        }
    }
    return maxLen
}

// Pattern 2: Concurrent safe data access
type SafeMap[K comparable, V any] struct {
    mu sync.RWMutex
    m  map[K]V
}
func (s *SafeMap[K, V]) Get(k K) (V, bool) {
    s.mu.RLock(); defer s.mu.RUnlock()
    v, ok := s.m[k]; return v, ok
}

// Pattern 3: Error group with context
g, ctx := errgroup.WithContext(context.Background())
results := make([]Result, len(inputs))
for i, input := range inputs {
    i, input := i, input
    g.Go(func() error {
        r, err := process(ctx, input)
        if err != nil { return err }
        results[i] = r
        return nil
    })
}
if err := g.Wait(); err != nil { return nil, err }
```

Q177

What is Go's HTTP client best practices?

```go
// Create once, share across goroutines (thread-safe)
var httpClient = &http.Client{
    Timeout: 30 * time.Second,
    Transport: &http.Transport{
        MaxIdleConnsPerHost:   100,
        IdleConnTimeout:       90 * time.Second,
        TLSHandshakeTimeout:   10 * time.Second,
        ResponseHeaderTimeout: 10 * time.Second,
    },
}

// Always drain body or connections won't be reused!
resp, err := httpClient.Do(req)
if err != nil { return err }
defer resp.Body.Close()
defer io.Copy(io.Discard, resp.Body)  // drain remaining body

// Context for cancellation
req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
resp, err := httpClient.Do(req)

// Retry client (using hashicorp/go-retryablehttp)
client := retryablehttp.NewClient()
client.RetryMax = 3
client.RetryWaitMin = 1 * time.Second
client.RetryWaitMax = 10 * time.Second
```

Q178

What is Go's io.ReadAll pitfalls?

```go
// io.ReadAll: reads entire body into memory
// Problem: no size limit → memory exhaustion attack

// BAD
data, _ := io.ReadAll(resp.Body)

// GOOD: limit body size
data, err := io.ReadAll(io.LimitReader(resp.Body, 10*1024*1024)) // 10MB limit

// For streaming: don't ReadAll, process incrementally
dec := json.NewDecoder(resp.Body)
for dec.More() {
    var item Item
    dec.Decode(&item)
    process(item)
}

// io.ReadAll vs ioutil.ReadAll:
// ioutil.ReadAll: deprecated (Go 1.16+)
// io.ReadAll: current (identical implementation)

// Temporary buffer (avoid allocation)
buf := bufPool.Get().(*bytes.Buffer)
buf.Reset()
defer bufPool.Put(buf)
io.Copy(buf, io.LimitReader(body, maxSize))
```

Q179

What is Go embed for static assets?

```go
//go:embed static
var staticFiles embed.FS

//go:embed templates/*.html
var templates embed.FS

//go:embed VERSION
var version string  // embed text file as string

// Serve embedded files
http.Handle("/", http.FileServer(http.FS(staticFiles)))

// Read embedded file
data, _ := staticFiles.ReadFile("static/index.html")

// Walk embedded directory
entries, _ := staticFiles.ReadDir("static")
for _, e := range entries {
    fmt.Println(e.Name(), e.IsDir())
}

// Use in tests (embed test fixtures)
//go:embed testdata/*.json
var testdata embed.FS

func TestParser(t *testing.T) {
    data, _ := testdata.ReadFile("testdata/input.json")
    result := Parse(data)
    // ...
}
```

Q180

What is Go standard library useful packages summary?

```go
Core:
  fmt:         formatted I/O
  os:          OS interface (files, env, signals)
  io:          I/O primitives (Reader, Writer, Copy)
  bufio:       buffered I/O
  bytes/strings: byte/string manipulation
  strconv:     type ↔ string conversion

Data:
  encoding/json: JSON marshal/unmarshal
  encoding/xml:  XML
  encoding/csv:  CSV
  encoding/base64: Base64
  encoding/binary: binary encoding

Network:
  net:         TCP/UDP/Unix sockets
  net/http:    HTTP client/server
  net/url:     URL parsing

Concurrency:
  sync:        Mutex, WaitGroup, Once, Pool, Map, Cond
  sync/atomic: atomic operations
  context:     cancellation, deadlines, values

Testing:
  testing:     test framework, benchmarks, fuzz testing
  net/http/httptest: HTTP testing utilities

Utilities:
  sort:        sorting (generic in Go 1.21+)
  math/rand:   pseudo-random
  crypto/rand: cryptographic random
  time:        time, duration, timer, ticker
  log/slog:    structured logging (Go 1.21+)
  embed:       embed files in binary (Go 1.16+)
```

Q181–Q200: Final Go Questions

| Q | Topic |

|---|---|

| Q181 | Go module graph and dependency resolution |

| Q182 | go:noescape and stack allocation hints |

| Q183 | Channel select fairness (uniform random) |

| Q184 | Generic type inference rules |

| Q185 | Go program startup sequence |

| Q186 | Testing with golden files (testdata/) |

| Q187 | go test -short flag usage |

| Q188 | Benchmark calibration (b.N) |

| Q189 | pprof block profile (lock contention) |

| Q190 | go tool trace scheduler analysis |

| Q191 | Unsafe string/bytes conversion (zero copy) |

| Q192 | Goroutine dump (SIGQUIT / /debug/pprof) |

| Q193 | Go 1.21+ slices and maps standard packages |

| Q194 | HTTP/2 in Go standard library |

| Q195 | Testing context cancellation |

| Q196 | Go workspace for monorepo |

| Q197 | Build constraints for test vs production |

| Q198 | go vet checks and common violations |

| Q199 | Production readiness for Go services |

| Q200 | Go interview quick reference card |

Q182

What is unsafe string/bytes zero-copy conversion?

```go
import "unsafe"
// Convert string to []byte without allocation (read-only!)
func StringToBytes(s string) []byte {
    return unsafe.Slice(unsafe.StringData(s), len(s))
}
// Convert []byte to string without allocation
func BytesToString(b []byte) string {
    return unsafe.String(unsafe.SliceData(b), len(b))
}
// Safe alternative (copies): []byte(s) or string(b)
// Only use unsafe when benchmarks prove it matters and slice is NOT modified
```

Q183

What is Go 1.21 slices package?

```go
import "slices"
// Sort
slices.Sort(nums)                        // sort in place
slices.SortFunc(users, func(a, b User) int { return cmp.Compare(a.Name, b.Name) })
// Search
idx, found := slices.BinarySearch(sorted, 42)
// Contains / Index
slices.Contains(s, "go")
slices.Index(s, "go")
// Reverse / Compact / Clip
slices.Reverse(s)
s = slices.Compact(s)                    // remove consecutive duplicates
s = slices.Clip(s)                       // reduce capacity to length
// Compare
slices.Equal(a, b)
slices.Compare(a, b)                     // lexicographic
```

Q184

What is Go 1.21 maps package?

```go
import "maps"
// Copy all entries
maps.Copy(dst, src)
// Delete matching keys
maps.DeleteFunc(m, func(k, v int) bool { return v < 0 })
// Collect keys/values (Go 1.23+)
for k, v := range maps.All(m) { fmt.Println(k, v) }
keys := slices.Collect(maps.Keys(m))
vals := slices.Collect(maps.Values(m))
// Clone
clone := maps.Clone(m)
// Equal
maps.Equal(a, b)
```

Q185

What is Go's cmp package?

```go
import "cmp"
// Compare two ordered values
cmp.Compare(1, 2)    // -1
cmp.Compare(2, 2)    //  0
cmp.Compare(3, 2)    //  1
// Or / Zero
cmp.Or(a, b, c)      // returns first non-zero value
// Use in sort.Func callbacks
slices.SortFunc(items, func(a, b Item) int {
    if n := cmp.Compare(a.Priority, b.Priority); n != 0 { return n }
    return cmp.Compare(a.Name, b.Name)
})
```

Q186

What is Go HTTP/2 server push?

```go
// HTTP/2 server push: proactively send assets before client asks
http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
    if pusher, ok := w.(http.Pusher); ok {
        pusher.Push("/static/style.css", &http.PushOptions{
            Header: http.Header{"Content-Type": []string{"text/css"}},
        })
    }
    fmt.Fprintln(w, "Hello HTTP/2!")
})
// Enable HTTP/2 (automatic with TLS in Go)
srv := &http.Server{Addr: ":443"}
srv.ListenAndServeTLS("cert.pem", "key.pem")
// Note: browser support for push is declining (Chrome removed in 2022)
// Use preload hints instead: Link: </style.css>; rel=preload; as=style
```

Q187

What is Go's testing/slog structured logging?

```go
import "log/slog"  // Go 1.21+
// Default text handler
slog.Info("user created", "id", 42, "name", "Alice")
// JSON handler
logger := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelDebug,
}))
logger.Info("request", "method", "GET", "path", "/api/users", "duration_ms", 12)
// With context
logger = logger.With("service", "auth", "version", "1.2.0")
logger.Error("auth failed", "user_id", 42, "err", err)
// Output: {"time":"...","level":"INFO","msg":"request","service":"auth",...}
```

Q188

What is table-driven test best practices?

```go
func TestAdd(t *testing.T) {
    tests := []struct {
        name    string
        a, b    int
        want    int
        wantErr bool
    }{
        {"positive", 1, 2, 3, false},
        {"negative", -1, -2, -3, false},
        {"overflow", math.MaxInt64, 1, 0, true},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            t.Parallel()  // run subtests in parallel
            got, err := Add(tt.a, tt.b)
            if (err != nil) != tt.wantErr {
                t.Errorf("Add() error = %v, wantErr %v", err, tt.wantErr)
            }
            if !tt.wantErr && got != tt.want {
                t.Errorf("Add() = %v, want %v", got, tt.want)
            }
        })
    }
}
```

Q189

What is Go workspace mode (go work)?

```bash
# go.work: develop multiple modules together without publishing
go work init
go work use ./service-a ./service-b ./shared-lib

# go.work file:
go 1.22
use (
    ./service-a
    ./service-b
    ./shared-lib
)
# service-a's go.mod can require shared-lib and workspace resolves it locally
# Without workspace: need to use replace directives in each go.mod

# Build in workspace mode (automatic when go.work exists):
go build ./...
go test ./...

# Disable workspace for CI (use published versions):
GOWORK=off go build ./...

# Use case: monorepo with multiple Go modules
```

Q190

What is Go's expvar package?

```go
import "expvar"
import _ "net/http/pprof"  // register /debug/pprof
// expvar: exported variables over HTTP at /debug/vars
var (
    requestsTotal = expvar.NewInt("requests_total")
    activeConns   = expvar.NewInt("active_connections")
    cacheHitRate  = expvar.NewFloat("cache_hit_rate")
)
func handler(w http.ResponseWriter, r *http.Request) {
    requestsTotal.Add(1)
    // ...
}
// GET /debug/vars → JSON with all expvars + memstats
// Use: quick metrics without full Prometheus setup
// Production: prefer Prometheus for dashboards/alerting
```

Q191

What is go vet and staticcheck?

```bash
# go vet: built-in static analysis
go vet ./...
# Catches: unreachable code, suspicious printf formats, struct tag errors,
#          mutex copied by value, shadowed variables, etc.

# Common vet errors:
# printf: fmt.Printf("%d", "string")  -- type mismatch
# copylocks: sync.Mutex passed by value (should be pointer)
# structtag: `json:"name,omitempty"` misspelled

# staticcheck: comprehensive linter (superset of vet)
go install honnef.co/go/tools/cmd/staticcheck@latest
staticcheck ./...
# Catches: deprecated API usage, unnecessary conversions,
#          unreachable code, incorrect error handling patterns

# golangci-lint: meta-linter (runs many linters)
golangci-lint run
# Config: .golangci.yml
# Include: errcheck, govet, ineffassign, staticcheck, gosec, revive

# Common in CI:
make lint  # typically runs golangci-lint with project config
```

Q192

What is Go's build tags for conditional compilation?

```go
//go:build linux && amd64
package main
// Only compiled on Linux AMD64

//go:build !integration
package mypackage
// Skipped when: go test -tags=integration ./...

// Common tags:
//go:build ignore          // never compiled (example files)
//go:build go1.21          // only Go 1.21+
//go:build cgo             // only when CGO enabled
//go:build windows || darwin

// Run integration tests:
go test -tags=integration ./...

// Platform-specific files (auto-detected by filename):
// os_linux.go     → only on linux
// os_darwin.go    → only on macOS
// os_windows.go   → only on windows
// net_cgo.go      → only when CGO enabled
```

Q193

What is Go's sync.Pool lifetime guarantees?

```go
// sync.Pool: reuse objects, reduce GC pressure
// WARNING: GC can drain pool at any time (not a cache!)
var bufPool = sync.Pool{
    New: func() interface{} { return new(bytes.Buffer) },
}
func process(data []byte) {
    buf := bufPool.Get().(*bytes.Buffer)
    buf.Reset()  // MUST reset before use
    defer bufPool.Put(buf)
    buf.Write(data)
    // use buf...
}
// Pool cleared every GC cycle (typically every 2 minutes or under memory pressure)
// Pool only useful for: short-lived objects, high allocation rate
// Not for: connections (use custom pool with health checks)
// sync.Pool is goroutine-safe (uses per-P local pools)
```

Q194

What is context propagation in Go microservices?

```go
// Pass context everywhere: timeout, cancellation, tracing
func (s *Service) GetOrder(ctx context.Context, id string) (*Order, error) {
    // Propagates: deadline, cancellation, trace span
    order, err := s.db.QueryRow(ctx, "SELECT ... WHERE id=$1", id)
    if err != nil { return nil, err }
    user, err := s.userSvc.GetUser(ctx, order.UserID)  // same context
    if err != nil { return nil, err }
    return &Order{...}, nil
}

// Request-scoped values: use typed keys
type ctxKey int
const (
    ctxRequestID ctxKey = iota
    ctxUserID
)
ctx = context.WithValue(ctx, ctxRequestID, requestID)
requestID := ctx.Value(ctxRequestID).(string)

// Timeout per-operation (not just per-request)
dbCtx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()
s.db.QueryRow(dbCtx, ...)

// Context in gRPC: automatic propagation via metadata
// Use: google.golang.org/grpc/metadata for custom values
```

Q195

What is Go's net.Conn and custom protocol?

```go
// Implement custom binary protocol over TCP
type Packet struct {
    Length  uint32
    Type    uint8
    Payload []byte
}
func ReadPacket(conn net.Conn) (*Packet, error) {
    header := make([]byte, 5)  // 4 bytes length + 1 byte type
    if _, err := io.ReadFull(conn, header); err != nil { return nil, err }
    length := binary.BigEndian.Uint32(header[:4])
    if length > 1<<20 { return nil, errors.New("packet too large") }
    payload := make([]byte, length)
    if _, err := io.ReadFull(conn, payload); err != nil { return nil, err }
    return &Packet{Length: length, Type: header[4], Payload: payload}, nil
}
func WritePacket(conn net.Conn, p *Packet) error {
    buf := make([]byte, 5+len(p.Payload))
    binary.BigEndian.PutUint32(buf[:4], uint32(len(p.Payload)))
    buf[4] = p.Type
    copy(buf[5:], p.Payload)
    _, err := conn.Write(buf)
    return err
}
```

Q196

What is Go's math/big for arbitrary precision?

```go
import "math/big"
// Large integers
n := new(big.Int).SetString("12345678901234567890", 10)
m := new(big.Int).SetInt64(99999999999)
result := new(big.Int).Mul(n, m)
fmt.Println(result.String())
// Fibonacci
func fibBig(n int) *big.Int {
    a, b := big.NewInt(0), big.NewInt(1)
    for i := 0; i < n; i++ { a, b = b, new(big.Int).Add(a, b) }
    return a
}
// Modular exponentiation (crypto)
base, exp, mod := big.NewInt(2), big.NewInt(100), big.NewInt(1000000007)
result2 := new(big.Int).Exp(base, exp, mod)  // 2^100 mod 10^9+7
// Floating point
f := new(big.Float).SetPrec(200).SetFloat64(math.Pi)
```

Q197

What is Go production service checklist?

```go
Build:
  ■ CGO_ENABLED=0 for static binary (no glibc dependency)
  ■ -ldflags="-s -w" (strip debug, reduce binary size)
  ■ Version/commit embedded: -ldflags="-X main.version=$(git rev-parse --short HEAD)"
  ■ go mod tidy before build (clean dependencies)
  ■ govulncheck: scan for known vulnerabilities

Runtime:
  ■ GOMAXPROCS set (default: num CPUs; tune for containers)
  ■ GOMEMLIMIT set (Go 1.19+: soft memory limit to avoid OOM)
  ■ Graceful shutdown: signal.NotifyContext + server.Shutdown
  ■ /healthz and /readyz endpoints
  ■ Structured logging (slog or zap)
  ■ Prometheus metrics exposed

Testing:
  ■ Unit tests: go test -race ./...
  ■ Integration tests: testcontainers or docker-compose
  ■ Benchmark: go test -bench=. (regression testing)
  ■ Fuzz: go test -fuzz (for parsers, handlers)
  ■ Coverage: go test -coverprofile=coverage.out

Observability:
  ■ OpenTelemetry traces
  ■ pprof endpoint: /debug/pprof (internal only)
  ■ Structured logging with request IDs
  ■ Panic recovery middleware (recover → 500 + log)
```

Q198

What is GOMEMLIMIT and GOGC tuning?

```bash
# GOGC: GC target percentage (default 100)
# 100 = trigger GC when heap doubles since last collection
GOGC=200  # less frequent GC, higher memory usage
GOGC=50   # more frequent GC, lower memory usage
GOGC=off  # disable GC (only for batch jobs!)

# GOMEMLIMIT (Go 1.19+): soft memory limit
# GC becomes more aggressive when approaching limit
# Prevents OOM kills in memory-constrained environments
GOMEMLIMIT=1GiB  # or runtime/debug.SetMemoryLimit(1 << 30)

# Tuning for containers:
# Set GOMEMLIMIT to 90% of container memory limit
# Example: 2GB container → GOMEMLIMIT=1800MiB

# Ballast trick (pre-1.19, now obsolete):
// ballast := make([]byte, 1<<30)  // 1GB ballast tricked GC to run less often
// Replaced by: GOMEMLIMIT

# Profile GC: GODEBUG=gctrace=1
# Output: gc N @Ts %: ...+... ms clock, ...+.../...+... ms cpu, ...->...->... MB, ...
```

Q199

What is Go's new/make/var initialization best practices?

```go
// var: zero value (good for sync.Mutex, error, basic types)
var mu sync.Mutex     // correct — zero value is valid unlocked mutex
var err error         // correct — nil
var count int         // correct — 0

// new: allocate + zero, returns pointer
p := new(sync.Mutex)  // equivalent to var mu sync.Mutex; &mu

// make: allocate + initialize (slices, maps, channels)
s := make([]int, 0, 10)          // len=0, cap=10
m := make(map[string]int)        // initialized empty map
ch := make(chan int, 100)         // buffered channel

// Literal: structs, slices, maps
user := User{Name: "Alice", Age: 30}
nums := []int{1, 2, 3}
m2 := map[string]int{"a": 1, "b": 2}

// Never: var m map[string]int; m["k"] = 1  → PANIC (nil map write)
// Always make maps before writing!

// nil slice is valid (can append, len=0, cap=0):
var s2 []int            // nil slice, valid
s2 = append(s2, 1, 2)  // works fine
```

Q200

What is Go quick reference for interviews?

```go
Data Structures:
  Array:        [N]T                O(1) access, fixed size
  Slice:        []T                 O(1) amortized append, dynamic
  Map:          map[K]V             O(1) avg, hash map
  Channel:      chan T              goroutine communication
  Heap:         container/heap      O(log N) push/pop
  List:         container/list      O(1) insert/delete
  Ring:         container/ring      circular doubly-linked

Concurrency:
  goroutine:    go f()              lightweight (2KB stack)
  sync.Mutex:   mutual exclusion
  sync.RWMutex: multiple readers OR one writer
  sync.WaitGroup: wait for goroutines
  sync.Once:    execute once
  sync.Pool:    object pool (GC-able)
  atomic:       lock-free int/uint/pointer ops
  errgroup:     goroutines with error propagation

Interfaces:
  io.Reader:    Read(p []byte) (n int, err error)
  io.Writer:    Write(p []byte) (n int, err error)
  error:        Error() string
  fmt.Stringer: String() string
  context.Context: cancellation + deadline + values

Patterns:
  Fan-out:      1 channel → N workers
  Fan-in:       N channels → 1 channel
  Pipeline:     stages connected by channels
  Done channel: cancel propagation
  Semaphore:    buffered channel as limiter
  Worker pool:  fixed N goroutines process jobs
```